WEB & APP DEVELOPMENT

Rapid Prototyping (FastAPI, Streamlit & Gradio)

Ship a working demo in days, not quarters: FastAPI, Streamlit and Gradio end to end, demo-proofing, lightweight auth, Docker deployment and the graduation path to production.

rapid-prototyping fastapi streamlit gradio python uv pydantic sse-streaming docker demo-engineering rag-demo prototype-debt oauth2-proxy fde

Study Pack · Comprehensive reference & interview prep · ~94 min read · 20,605 words

CONTENTS

1. The Demo-in-a-Day Doctrine
2. Choosing the Stack
3. Python Project Setup That Does Not Waste an Hour
4. FastAPI Fundamentals
5. FastAPI for AI Applications
6. Streamlit Deep Dive
7. Gradio Deep Dive
8. Worked Example: Document Q&A Over a Customer's PDFs
9. Making a Prototype Demo-Proof
10. Lightweight Auth for Prototypes
11. Packaging and Deployment
12. Observability and Feedback in a Prototype
13. Performance Tricks
14. The Graduation Path
15. Anti-Patterns and How Prototypes Fail
16. Interview Questions
17. Build-a-Demo Lab Plan and Checklists

The Demo-in-a-Day Doctrine

A Forward Deployed Engineer's core move is to compress the distance between a customer saying "could it do X?" and seeing X working on their own data. A deck describing a capability generates a follow-up meeting. A working thing that answers a question they actually have generates a budget. The entire discipline of rapid prototyping is about making that second outcome achievable within days rather than quarters, without writing code you will be ashamed of.

1-3 days	<500	80/20	0
TARGET TIME TO FIRST WORKING DEMO	LINES FOR MOST USEFUL PROTOTYPES	SHOW THE 20% OF FEATURES THAT CARRY 80% OF VALUE	ACCEPTABLE LIVE FAILURES IN FRONT OF A CUSTOMER

Why the working prototype wins

Three things happen when a customer uses a prototype rather than hearing about it, and none of them happen in a slide review.

REQUIREMENTS BECOME REAL

In a workshop people describe the system they imagine. In front of a prototype they say "no, we'd never type the order number, we'd paste the whole email" — a sentence worth four weeks of guesswork. Prototypes extract tacit requirements that no amount of interviewing surfaces.

THE SCEPTICS MOVE

Every room has someone who believes it cannot work on their messy data. You cannot argue them out of it. You can run it on their messy data in front of them. Their objection becomes a specific, solvable gap instead of a general doubt.

YOU FIND THE REAL BLOCKERS EARLY

Building even a toy version forces you to get data access, discover that the API needs a VPN, learn that the PDFs are scanned images, and find out that half the records have no customer id. These are the things that kill projects in month four. Find them in week one.

The rules of prototype scope

1. **One user, one task, one happy path.**
   Pick the single most valuable workflow and make it genuinely excellent. Three mediocre workflows demo worse than one that works.

2. **Real data, always.**
   A prototype on synthetic data proves nothing to the person whose data is messy. Get a sample — masked, sampled, whatever it takes — but make it theirs. The moment they see their own product names on screen, the conversation changes.

3. **Fake the expensive parts deliberately and honestly.**
   Hardcode the auth. Stub the write-back to SAP. Use a CSV instead of the live feed. Then say so out loud during the demo: "the write-back is stubbed; it's two days of work once we have the API credentials." Honesty about stubs buys enormous trust.

4. **No infrastructure you do not need today.**
   No Kubernetes, no message queue, no microservices, no auth service, no CI pipeline. A single process on one box is correct for a prototype and you should be comfortable defending that.

5. **Timebox hard and ship the timebox.**
   Three days means three days. If something is not done, cut scope, never quality of the demo path. A rough edge off the happy path is invisible; a crash on the happy path is the only thing anyone remembers.

6. **Write down everything you deferred.**
   A prototype debt register, started on day one, is the artefact that lets you answer "how long to productionise?" with a number instead of a shrug.

The prototype trap

The demo goes well, the customer says "great, can we just put that live next week?", and six months later you are still running a Streamlit app with a hardcoded API key serving forty people. The defence is not refusing — it is having stated from the first demo, in writing, what the graduation path costs. "This is a prototype. To run it for real needs authentication, tests, a database instead of local files, and deployment into your tenancy: roughly four weeks." Say that in demo one, repeat it in demo two, and put it in the summary email. Then the decision to ship it anyway is theirs, made with information.

What a prototype is not

A prototype	A product
Proves a capability is possible on real data	Serves many users reliably over years
One happy path, built fast	All paths, including the nasty ones
Errors shown as a friendly message to the demo operator	Errors handled, logged, alerted, retried
Secrets in a `.env` file you control	Secrets in a managed vault with rotation
State in memory or a local file	State in a database with migrations and backups
Run by you, on your terms, in a meeting	Run by their team, at 3 a.m., without you
Disposable — you expect to rewrite it	Maintained — you expect to extend it

Build it to be thrown away, but make it easy not to

These two are not in conflict if you keep the business logic in plain, testable Python functions and keep the UI layer thin. Then "productionising" means swapping the UI and the storage, not rewriting the thinking. The single highest-leverage structural decision in a prototype is: **no business logic inside a Streamlit callback or a FastAPI route handler.**

Choosing the Stack

The wrong framework costs you a day of fighting the tool instead of solving the problem. The choice is almost mechanical once you know who will use the thing, whether it needs

to be embedded in something else, and whether it has to survive beyond the demo.

The decision table

If…	Use	Because
A data/ML demo for 1-20 internal users, you own the whole UI	**Streamlit**	Fastest path from script to shareable app; layout, widgets, charts, chat all built in
A model demo you want to share as a link, or embed in a page	**Gradio**	Purpose-built for model I/O; `share=True` gives a public tunnel in one line; native on HF Spaces
The customer's own frontend must call it, or it is a service	**FastAPI**	Real HTTP API, OpenAPI docs, typed validation, streaming, async
You need a polished, branded UI the customer's designers specified	**FastAPI + React/Next.js**	Full control of the UI; costs 3-5x the frontend time
The audience is data scientists and the artefact is the analysis	**Jupyter / Marimo**	Narrative plus code; nobody needs a web app to read a notebook
It is a form-plus-workflow with no custom logic	**No-code (Power Apps, Retool, Budibase)**	Their IT already supports it; you are not the bottleneck
An internal tool with CRUD over a database	**Retool / Appsmith / Streamlit + SQLModel**	Do not hand-build CRUD screens in 2026
It must run entirely inside their network with no internet	**FastAPI + Streamlit in one Docker Compose**	Two containers, no cloud dependencies, portable to any VM

Streamlit vs Gradio vs FastAPI, honestly

	Streamlit	Gradio	FastAPI
Mental model	Script re-runs top to bottom on every interaction	Event handlers bound to components	Request → handler → response
Time to a chat UI	~20 lines	~5 lines (`ChatInterface`)	You build the UI yourself
Layout control	Good (columns, tabs, containers, sidebar)	Good with Blocks, awkward with Interface	N/A
Multi-page	Native	Possible but clumsy	Routers
Custom styling	Theming + CSS injection; fights you past a point	Theming + CSS; similar limits	Unlimited
Auth	None built in (OIDC in recent versions / Community Cloud)	`auth=` tuple for basic gating	You implement it properly
Concurrency	One session per browser tab; shared process	Queue with configurable concurrency	Full async, workers, proper control
Used by a machine	No	Yes (it exposes an API)	Yes, that is the point
Graduates to production	Rarely well	Rarely well	Yes, directly

✓ **The default that is right most often**

FastAPI for the logic, Streamlit for the face. The backend holds everything real — retrieval, model calls, business rules — behind typed endpoints that can be tested with `pytest` and called by anything. The frontend is a thin client. When the customer says "actually we want this inside our portal", you throw away the Streamlit app and keep 90% of the work. When they say "can you just run it as a batch job", same answer. This split costs you about two extra hours on day one and saves weeks later.

When to say no to a prototype

- [ ] The data needed does not exist yet and will not within the engagement.
- [ ] Legal has not cleared sending the data to the model provider you planned to use, and there is no on-prem alternative in scope.
- [ ] The real question is organisational ("will the team adopt this?") and a prototype cannot answer it.
- [ ] They want a production system and are using the word "prototype" to avoid a procurement process. Clarify before you build.
- [ ] The demo audience is three levels above anyone who has used the current process — build it, but insist a real user is in the room.

Python Project Setup That Does Not Waste an Hour

Every hour spent on environment problems is an hour not spent on the demo, and environment problems compound: the thing works on your machine, then not on the customer's VM, then not in the container. Fix this once with a layout you can reproduce from muscle memory and you will never think about it again.

Use uv

`uv` replaces pip, virtualenv, pip-tools and most of poetry's role, and it is dramatically faster — resolving and installing a FastAPI + LangChain + pandas stack typically drops from minutes to seconds. Speed matters here because you will rebuild environments many times a day on a customer laptop.

BASH

```
# Install uv (one time)
curl -LsSf https://astral.sh/uv/install.sh | sh
# Windows: powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

# New project
uv init customer-demo && cd customer-demo
uv python pin 3.12                      # writes .python-version; pins the interpreter

# Add dependencies — resolves and installs into .venv automatically
uv add fastapi "uvicorn[standard]" pydantic pydantic-settings httpx
uv add streamlit pandas python-multipart
uv add --dev pytest pytest-asyncio ruff mypy

# Run anything inside the environment without activating it
uv run uvicorn app.main:app --reload
uv run streamlit run ui/app.py
uv run pytest -q

# Reproduce exactly elsewhere (uses uv.lock)
uv sync --frozen

# One-off tools without polluting the project
uvx ruff check .
```

Commit the lockfile

`uv.lock` (or `poetry.lock`, or a `requirements.txt` produced by `uv pip compile`) is what makes the demo reproducible on the customer's machine. The classic prototype failure is arriving on site, running `pip install -r requirements.txt` with unpinned versions, getting a six-month-newer library with a breaking change, and spending the first hour of your demo day debugging an import. Pin everything.

pyproject.toml

TEXT

```
[project]
name = "customer-demo"
version = "0.1.0"
requires-python = "≥3.11"
dependencies = [
    "fastapi≥0.115",
    "uvicorn[standard]≥0.32",
    "pydantic≥2.9",
    "pydantic-settings≥2.6",
    "httpx≥0.27",
    "python-multipart≥0.0.12",
    "streamlit≥1.40",
    "pandas≥2.2",
]

[dependency-groups]
dev = ["pytest≥8.3", "pytest-asyncio≥0.24", "ruff≥0.8", "mypy≥1.13"]

[tool.ruff]
line-length = 100
target-version = "py312"

[tool.ruff.lint]
select = ["E", "F", "I", "UP", "B", "SIM"]

[tool.pytest.ini_options]
asyncio_mode = "auto"
testpaths = ["tests"]
```

Project layout for a prototype that might graduate

TEXT

```
customer-demo/
  pyproject.toml
  uv.lock
  .env.example                # committed — documents required vars, NO values
  .env                        # gitignored — real values
  .gitignore
  README.md                   # how to run it, in 5 lines
  docker-compose.yml
  Makefile
  app/                        # FastAPI backend
    __init__.py
    main.py                   # app factory + router registration ONLY
    config.py                 # pydantic-settings
    deps.py                   # dependency-injection providers
    routers/
      health.py
      ask.py
      documents.py
    services/                 # ALL business logic lives here. No framework imports.
      retrieval.py
      llm.py
      ingest.py
    models/                   # pydantic request/response schemas
      schemas.py
  ui/                         # Streamlit frontend
    app.py
    api_client.py             # the ONLY file that knows the backend URL
    components.py
    pages/
      1_Ask.py
      2_Documents.py
      3_Feedback.py
  tests/
    test_services.py          # these test real logic and run in 2 seconds
    test_api.py
  data/
    samples/                  # seeded demo data, committed if not sensitive
  scripts/
    seed.py
    ingest_pdfs.py
```

> ⚠ **The one rule that makes the layout worth it**
>
> `app/services/*.py` must not import FastAPI, and must not import Streamlit. They take plain arguments and return plain objects. This means you can test them without a server, call them from a script, call them from a notebook, and reuse them unchanged when the UI is replaced. Prototypes that violate this rule cannot be graduated; they can only be rewritten.

Configuration and secrets

PYTHON

```python
# app/config.py
from functools import lru_cache
from pydantic import Field, SecretStr
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
        case_sensitive=False,
    )

    # Required — the app refuses to start without them, with a clear error
    openai_api_key: SecretStr
    vector_store_path: str

    # Optional with sane defaults
    app_name: str = "Customer Document Assistant"
    model_name: str = "gpt-4o-mini"
    embedding_model: str = "text-embedding-3-small"
    max_context_chunks: int = Field(default=6, ge=1, le=20)
    request_timeout_s: float = Field(default=60.0, gt=0)
    demo_mode: bool = False                    # serves canned answers; see "demo-proo
    log_level: str = "INFO"
    allowed_origins: list[str] = ["http://localhost:8501"]

@lru_cache
def get_settings() -> Settings:
    return Settings()                          # cached; validated once at startup
```

BASH

```
# .env.example  - COMMIT THIS. It is the contract for whoever runs the app next.
OPENAI_API_KEY=sk-replace-me
VECTOR_STORE_PATH=./data/index
MODEL_NAME=gpt-4o-mini
MAX_CONTEXT_CHUNKS=6
DEMO_MODE=false
LOG_LEVEL=INFO
```

The `.env` that ended up on GitHub

Add `.env` to `.gitignore` before your first commit, not after. If a key does leak, rotating it is the only fix — deleting the commit does not help, because the history and any fork retain it, and scrapers find committed keys within minutes. On a customer engagement a leaked key is not just embarrassing, it is a reportable security incident. Use `SecretStr` so the value does not appear in logs or tracebacks, and run `gitleaks` or `detect-secrets` in a pre-commit hook.

A Makefile is the fastest documentation

BASH

```
.PHONY: install api ui dev test lint fmt seed docker

install:
	uv sync

api:
	uv run uvicorn app.main:app --reload --port 8000

ui:
	uv run streamlit run ui/app.py --server.port 8501

dev:
	uv run honcho start          # runs api and ui together (Procfile)

test:
	uv run pytest -q

lint:
	uv run ruff check . && uv run mypy app

fmt:
	uv run ruff format . && uv run ruff check --fix .

seed:
	uv run python scripts/seed.py

docker:
	docker compose up --build
```

FastAPI Fundamentals

FastAPI gives you a typed, validated, self-documenting HTTP API with very little ceremony, and it is the right backend for anything a customer's systems will eventually call. The parts worth knowing well are Pydantic validation, dependency injection, and the async model — because those three determine whether your prototype survives being pointed at by something other than your browser.

The minimum real app

PYTHON

```python
# app/main.py
import logging
from contextlib import asynccontextmanager

from fastapi import FastAPI, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import JSONResponse

from app.config import get_settings
from app.routers import ask, documents, health
from app.services.retrieval import VectorIndex

logging.basicConfig(level=get_settings().log_level,
                    format="%(asctime)s %(levelname)s %(name)s %(message)s")
log = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    """Load expensive things ONCE at startup, not per request.
    This is the difference between a 40 ms demo and a 4 s demo."""
    s = get_settings()
    log.info("loading vector index from %s", s.vector_store_path)
    app.state.index = VectorIndex.load(s.vector_store_path)
    log.info("index ready: %d chunks", app.state.index.size)
    yield
    log.info("shutting down")
    app.state.index.close()

def create_app() -> FastAPI:
    s = get_settings()
    app = FastAPI(
        title=s.app_name,
        version="0.1.0",
        description="Prototype document assistant. Not production hardened.",
        lifespan=lifespan,
    )
    app.add_middleware(
        CORSMiddleware,
        allow_origins=s.allowed_origins,       # never ["*"] with credentials
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )
    app.include_router(health.router)
    app.include_router(ask.router,       prefix="/api/v1")
    app.include_router(documents.router, prefix="/api/v1")

    @app.exception_handler(Exception)
```

```
    async def unhandled(request: Request, exc: Exception):
        log.exception("unhandled error on %s %s", request.method, request.url.path)
        return JSONResponse(
            status_code=500,
            content={"error": "internal_error",
                     "detail": "Something went wrong. The team has been notified.",
                     "request_id": request.headers.get("x-request-id", "-")},
        )
    return app

app = create_app()
```

Parameters and Pydantic v2 models

PYTHON

```python
# app/models/schemas.py
from datetime import datetime
from enum import Enum
from typing import Annotated, Literal

from pydantic import BaseModel, Field, field_validator, model_validator

class Scope(str, Enum):
    all = "all"
    policies = "policies"
    contracts = "contracts"

class AskRequest(BaseModel):
    question: Annotated[str, Field(min_length=3, max_length=2000,
                                   examples=["What is the notice period for a fixed-te
    scope: Scope = Scope.all
    top_k: Annotated[int, Field(ge=1, le=20)] = 6
    temperature: Annotated[float, Field(ge=0.0, le=1.0)] = 0.0
    include_sources: bool = True

    @field_validator("question")
    @classmethod
    def strip_and_check(cls, v: str) -> str:
        v = v.strip()
        if not v:
            raise ValueError("question cannot be blank")
        return v

class Citation(BaseModel):
    document_id: str
    title: str
    page: int | None = None
    snippet: str
    score: float

class AskResponse(BaseModel):
    answer: str
    citations: list[Citation] = []
    model: str
    latency_ms: int
    request_id: str
    truncated: bool = False

class DateRange(BaseModel):
    start: datetime
    end: datetime

    @model_validator(mode="after")
```

```
    def ordered(self):
        if self.start ≥ self.end:
            raise ValueError("start must be before end")
        return self
```

PYTHON

```python
# app/routers/ask.py
import time, uuid
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException, Query, status

from app.deps import get_index, get_llm, verify_api_key
from app.models.schemas import AskRequest, AskResponse
from app.services.retrieval import VectorIndex
from app.services.llm import LLMClient, LLMTimeout, LLMRefusal

router = APIRouter(tags=["ask"], dependencies=[Depends(verify_api_key)])

@router.post("/ask", response_model=AskResponse,
             responses={422: {"description": "Validation error"},
                        503: {"description": "Model unavailable"}})
async def ask(
    req: AskRequest,
    index: Annotated[VectorIndex, Depends(get_index)],
    llm: Annotated[LLMClient, Depends(get_llm)],
) -> AskResponse:
    started = time.perf_counter()
    request_id = str(uuid.uuid4())[:8]

    chunks = index.search(req.question, k=req.top_k,
                          filters={"scope": req.scope.value} if req.scope != "all" els
    if not chunks:
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail="No relevant documents found. Try rephrasing or widening the scope.
        )

    try:
        result = await llm.answer(req.question, chunks, temperature=req.temperature)
    except LLMTimeout:
        raise HTTPException(status.HTTP_504_GATEWAY_TIMEOUT,
                            detail="The model took too long. Try a shorter question.")
    except LLMRefusal as e:
        raise HTTPException(status.HTTP_422_UNPROCESSABLE_ENTITY, detail=str(e))

    return AskResponse(
        answer=result.text,
        citations=result.citations if req.include_sources else [],
        model=llm.model_name,
        latency_ms=int((time.perf_counter() - started) * 1000),
        request_id=request_id,
        truncated=result.truncated,
    )
```

```python
# Path and query parameters, typed and validated automatically
@router.get("/documents/{document_id}/chunks")
async def list_chunks(
    document_id: str,
    index: Annotated[VectorIndex, Depends(get_index)],
    limit: Annotated[int, Query(ge=1, le=200)] = 50,
    offset: Annotated[int, Query(ge=0)] = 0,
):
    return index.chunks_for(document_id, limit=limit, offset=offset)
```

Dependency injection

FastAPI's `Depends` is the mechanism that keeps route handlers thin and makes everything testable. A dependency is just a callable whose result is injected; it can be cached, nested, and overridden in tests.

PYTHON

```python
# app/deps.py
import secrets
from typing import Annotated

from fastapi import Depends, Header, HTTPException, Request, status

from app.config import Settings, get_settings
from app.services.llm import LLMClient
from app.services.retrieval import VectorIndex

SettingsDep = Annotated[Settings, Depends(get_settings)]

def get_index(request: Request) -> VectorIndex:
    """Reads the singleton created in lifespan. No per-request loading."""
    return request.app.state.index

def get_llm(settings: SettingsDep) -> LLMClient:
    return LLMClient(api_key=settings.openai_api_key.get_secret_value(),
                     model=settings.model_name,
                     timeout_s=settings.request_timeout_s)

def verify_api_key(settings: SettingsDep,
                   x_api_key: Annotated[str | None, Header()] = None) -> None:
    """Minimal gate for a prototype. compare_digest avoids timing leaks."""
    expected = settings.demo_api_key.get_secret_value()
    if not x_api_key or not secrets.compare_digest(x_api_key, expected):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, detail="Invalid or missing A
```

PYTHON

```python
# tests/test_api.py — dependency_overrides is why DI is worth the ceremony
import pytest
from fastapi.testclient import TestClient

from app.main import create_app
from app.deps import get_index, get_llm, verify_api_key

class FakeIndex:
    size = 3
    def search(self, q, k, filters=None):
        return [{"document_id": "d1", "title": "Policy", "snippet": "Notice is 30 days
                 "score": 0.91, "page": 2}]

class FakeLLM:
    model_name = "fake"
    async def answer(self, q, chunks, temperature=0.0):
        class R: text, citations, truncated = "Thirty days.", [], False
        return R()

@pytest.fixture
def client():
    app = create_app()
    app.dependency_overrides[get_index]       = lambda: FakeIndex()
    app.dependency_overrides[get_llm]         = lambda: FakeLLM()
    app.dependency_overrides[verify_api_key]  = lambda: None
    return TestClient(app)

def test_ask_returns_answer(client):
    r = client.post("/api/v1/ask", json={"question": "What is the notice period?"})
    assert r.status_code == 200
    assert r.json()["answer"] == "Thirty days."

def test_short_question_rejected(client):
    r = client.post("/api/v1/ask", json={"question": "a"})
    assert r.status_code == 422          # Pydantic min_length, no code of yours ran
```

These tests take two seconds and save the demo

With fakes injected, your whole API suite runs without network, keys or a model. You can run it on the customer's locked-down laptop, in their CI, on a plane. The temptation in a prototype is to skip tests entirely; writing just these three — happy path, validation failure, upstream failure — costs twenty minutes and catches the regressions that happen when you change something an hour before a demo.

Error handling that reads well

PYTHON

```python
from fastapi import Request, status
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse

class DomainError(Exception):
    """Base for errors we expect and can explain to a user."""
    status_code = status.HTTP_400_BAD_REQUEST
    code = "domain_error"
    def __init__(self, message: str, **ctx):
        super().__init__(message)
        self.message, self.ctx = message, ctx

class DocumentNotIndexed(DomainError):
    status_code = status.HTTP_409_CONFLICT
    code = "document_not_indexed"

class ContextTooLarge(DomainError):
    status_code = status.HTTP_413_REQUEST_ENTITY_TOO_LARGE
    code = "context_too_large"

def register_error_handlers(app):
    @app.exception_handler(DomainError)
    async def domain_handler(request: Request, exc: DomainError):
        return JSONResponse(
            status_code=exc.status_code,
            content={"error": exc.code, "detail": exc.message, "context": exc.ctx},
        )

    @app.exception_handler(RequestValidationError)
    async def validation_handler(request: Request, exc: RequestValidationError):
        # Default FastAPI output is verbose; flatten it for a UI to display.
        problems = [{"field": ".".join(str(p) for p in e["loc"][1:]), "message": e["ms
                     for e in exc.errors()]
        return JSONResponse(status_code=422,
                            content={"error": "validation_failed", "problems": problem
```

Async vs sync, and when it actually matters

This is the single most misunderstood part of FastAPI and it causes real demo failures.

Handler type	Where it runs	Consequence
`async def` with only `await`ed I/O	The event loop	Correct. Thousands of concurrent waits on one process.
`async def` containing a blocking call (`requests.get`, `time.sleep`, heavy CPU)	The event loop — and **blocks it**	**Disaster.** Every other request on that worker stalls. The classic cause of "the demo froze".
`def` (plain)	A threadpool (default 40 threads)	Safe for blocking I/O. Limited by threadpool size; GIL limits CPU work.

PYTHON

```python
import asyncio, time
import httpx, requests
from fastapi import APIRouter
from fastapi.concurrency import run_in_threadpool

router = APIRouter()

# WRONG: blocking library inside async - freezes the whole event loop
@router.get("/bad")
async def bad():
    r = requests.get("https://slow.example.com/data", timeout=30)   # blocks everythin
    return r.json()

# RIGHT (a): async client inside async
@router.get("/good-async")
async def good_async():
    async with httpx.AsyncClient(timeout=30) as c:
        r = await c.get("https://slow.example.com/data")
    return r.json()

# RIGHT (b): keep the blocking library, use a plain def - FastAPI threadpools it
@router.get("/good-sync")
def good_sync():
    return requests.get("https://slow.example.com/data", timeout=30).json()

# RIGHT (c): CPU-bound work must leave the event loop explicitly
@router.post("/embed")
async def embed(texts: list[str]):
    vectors = await run_in_threadpool(model.encode, texts)   # or a process pool
    return {"n": len(vectors)}

# Concurrency control: never let a demo fan out 500 model calls at once
SEMAPHORE = asyncio.Semaphore(8)

async def call_model(prompt: str) -> str:
    async with SEMAPHORE:
        async with httpx.AsyncClient(timeout=60) as c:
            r = await c.post(MODEL_URL, json={"prompt": prompt})
            r.raise_for_status()
            return r.json()["text"]

@router.post("/batch")
async def batch(prompts: list[str]):
    results = await asyncio.gather(*(call_model(p) for p in prompts),
                                   return_exceptions=True)
    ok  = [r for r in results if not isinstance(r, Exception)]
```

```
    err = [str(r) for r in results if isinstance(r, Exception)]
    return {"results": ok, "errors": err}
```

> ✕ **The demo that froze**
>
> A prototype used `async def` handlers with the synchronous OpenAI SDK. In testing — one user, one request at a time — it was fine. In the demo, the customer opened the app in three tabs, and every request queued behind the first blocking call. It looked like the model was broken. The rule: inside `async def`, everything slow must be `await`ed or pushed to a threadpool. If you are not sure, write `def` instead of `async def` — the threadpool is slower in theory and far safer in practice.

OpenAPI docs as a deliverable

`/docs` gives you an interactive Swagger UI and `/redoc` a readable reference, both generated from your type hints. This is genuinely useful at a customer: their integration engineer can try your API in a browser without reading your code, and you can hand them `/openapi.json` to generate a client.

PYTHON

```
app = FastAPI(
    title="Document Assistant API",
    version="0.1.0",
    summary="Ask questions over the policy corpus.",
    description=(
        "**Prototype.** Rate limited, no SLA.\n\n"
        "Auth: send `X-API-Key`. Ask your contact for a key."
    ),
    contact={"name": "Delivery team", "email": "team@example.com"},
    docs_url="/docs",
    redoc_url="/redoc",
    openapi_tags=[
        {"name": "ask", "description": "Question answering over indexed documents."},
        {"name": "documents", "description": "Upload, list and reindex documents."},
    ],
)

# Generate a typed client for them in one command:
#   npx @hey-api/openapi-ts -i http://localhost:8000/openapi.json -o ./client
# or for Python:
#   uvx openapi-python-client generate --url http://localhost:8000/openapi.json
```

FastAPI for AI Applications

AI workloads break the assumptions of a normal web API: responses take ten seconds instead of fifty milliseconds, users expect to see tokens as they arrive, uploads are large, some jobs run for minutes, and the upstream provider rate-limits you. These patterns are what separate an AI prototype that feels fast from one that feels broken.

Streaming responses (SSE)

Streaming is the single highest-impact UX change in an LLM app. Time-to-first-token of 400 ms with tokens flowing feels instant; a 9-second blank screen feels broken, even though total latency is identical.

PYTHON

```python
import json
from collections.abc import AsyncIterator

from fastapi import APIRouter
from fastapi.responses import StreamingResponse

router = APIRouter()

async def sse_events(question: str, llm, index) -> AsyncIterator[str]:
    """Server-Sent Events. Each message is 'data: <payload>\\n\\n'."""
    # 1. Tell the client what is happening before the slow part starts
    yield f"data: {json.dumps({'type': 'status', 'message': 'Searching documents ... '})

    chunks = await index.asearch(question, k=6)
    yield f"data: {json.dumps({'type': 'sources', 'sources': [c.brief() for c in chunk
    yield f"data: {json.dumps({'type': 'status', 'message': 'Generating answer ... '})}\

    # 2. Stream the tokens
    try:
        async for token in llm.astream(question, chunks):
            yield f"data: {json.dumps({'type': 'token', 'text': token})}\n\n"
    except Exception as e:
        yield f"data: {json.dumps({'type': 'error', 'message': str(e)})}\n\n"
        return

    # 3. Explicit terminator so the client knows it is complete, not truncated
    yield f"data: {json.dumps({'type': 'done'})}\n\n"

@router.post("/ask/stream")
async def ask_stream(req: AskRequest, llm=Depends(get_llm), index=Depends(get_index)):
    return StreamingResponse(
        sse_events(req.question, llm, index),
        media_type="text/event-stream",
        headers={
            "Cache-Control": "no-cache",
            "Connection": "keep-alive",
            "X-Accel-Buffering": "no",      # critical behind nginx, or it buffers eve
        },
    )
```

PYTHON

```python
# Consuming SSE from Python (e.g. your Streamlit client)
import httpx, json

def stream_answer(question: str, api_url: str, api_key: str):
    with httpx.Client(timeout=httpx.Timeout(300.0, connect=10.0)) as client:
        with client.stream("POST", f"{api_url}/api/v1/ask/stream",
                           json={"question": question},
                           headers={"X-API-Key": api_key}) as response:
            response.raise_for_status()
            for line in response.iter_lines():
                if not line or not line.startswith("data: "):
                    continue
                event = json.loads(line[6:])
                if event["type"] == "token":
                    yield event["text"]
                elif event["type"] == "error":
                    raise RuntimeError(event["message"])
                elif event["type"] == "done":
                    return
```

⚠ **Proxies will silently break your stream**

nginx buffers responses by default, so your beautifully streamed tokens arrive as one lump at the end. Set `X-Accel-Buffering: no` on the response and `proxy_buffering off;` in nginx. Azure Container Apps, Cloud Run and most load balancers also need the response to start within their initial-response timeout, which is why you emit a status event immediately rather than waiting for retrieval to finish. Test streaming through the real deployment path, never just on localhost.

File uploads

PYTHON

```python
import hashlib
from pathlib import Path

from fastapi import APIRouter, BackgroundTasks, File, HTTPException, UploadFile, statu

router = APIRouter()

ALLOWED = {"application/pdf", "text/plain", "text/markdown",
           "application/vnd.openxmlformats-officedocument.wordprocessingml.document"}
MAX_BYTES = 25 * 1024 * 1024
CHUNK = 1024 * 1024

@router.post("/documents", status_code=status.HTTP_202_ACCEPTED)
async def upload(background: BackgroundTasks, files: list[UploadFile] = File( ... )):
    if len(files) > 20:
        raise HTTPException(400, "Maximum 20 files per upload")

    accepted, rejected = [], []
    for f in files:
        if f.content_type not in ALLOWED:
            rejected.append({"name": f.filename, "reason": f"unsupported type {f.conte
            continue

        # Stream to disk - never f.read() a 25 MB file into memory per request
        dest = Path("data/uploads") / f"{hashlib.sha256(f.filename.encode()).hexdigest
        dest.parent.mkdir(parents=True, exist_ok=True)
        size, digest = 0, hashlib.sha256()
        with dest.open("wb") as out:
            while block := await f.read(CHUNK):
                size += len(block)
                if size > MAX_BYTES:
                    out.close(); dest.unlink(missing_ok=True)
                    rejected.append({"name": f.filename, "reason": "exceeds 25 MB"})
                    break
                digest.update(block)
                out.write(block)
            else:
                accepted.append({"name": f.filename, "bytes": size,
                                 "sha256": digest.hexdigest()[:16], "path": str(dest)}

    for doc in accepted:
        background.add_task(index_document, doc["path"], doc["sha256"])

    return {"accepted": accepted, "rejected": rejected,
            "message": f"{len(accepted)} file(s) queued for indexing"}
```

Background tasks and long-running jobs

`BackgroundTasks` runs after the response is sent, in the same process. That is perfect for a prototype and wrong for anything that must survive a restart. Know the boundary.

Mechanism	Survives restart?	Use for
`BackgroundTasks`	No	Indexing an uploaded file, sending a webhook, writing a log — prototype-scale work
`asyncio.create_task`	No; also no backpressure	Fire-and-forget within one request lifetime. Keep a reference or it may be garbage collected.
In-process job registry (dict + task)	No, but gives progress and cancellation	Multi-minute jobs in a demo where the user needs a progress bar
Celery / RQ / Dramatiq + Redis	Yes	When the customer will actually run it; adds a broker to operate
Cloud queue (SQS, Service Bus, Pub/Sub) + worker	Yes	Production graduation

PYTHON

```python
import asyncio, uuid
from dataclasses import dataclass, field
from datetime import datetime, timezone
from enum import Enum

from fastapi import APIRouter, HTTPException

router = APIRouter()

class JobState(str, Enum):
    queued = "queued"; running = "running"; done = "done"
    failed = "failed"; cancelled = "cancelled"

@dataclass
class Job:
    id: str
    state: JobState = JobState.queued
    progress: float = 0.0
    message: str = ""
    result: dict | None = None
    error: str | None = None
    created_at: datetime = field(default_factory=lambda: datetime.now(timezone.utc))

JOBS: dict[str, Job] = {}
TASKS: dict[str, asyncio.Task] = {}

async def run_ingest(job: Job, paths: list[str]) -> None:
    try:
        job.state = JobState.running
        total = len(paths)
        for i, p in enumerate(paths, start=1):
            await asyncio.sleep(0)                 # yield so cancellation can land
            await index_one(p)
            job.progress = i / total
            job.message = f"Indexed {i}/{total}: {p.split('/')[-1]}"
        job.state, job.result = JobState.done, {"indexed": total}
    except asyncio.CancelledError:
        job.state, job.message = JobState.cancelled, "Cancelled by user"
        raise
    except Exception as e:
        job.state, job.error = JobState.failed, str(e)

@router.post("/jobs/ingest")
async def start_ingest(paths: list[str]):
    job = Job(id=str(uuid.uuid4())[:8])
    JOBS[job.id] = job
    TASKS[job.id] = asyncio.create_task(run_ingest(job, paths))
    return {"job_id": job.id}
```

```python
@router.get("/jobs/{job_id}")
async def job_status(job_id: str):
    if job_id not in JOBS:
        raise HTTPException(404, "Unknown job")
    j = JOBS[job_id]
    return {"id": j.id, "state": j.state, "progress": round(j.progress, 3),
            "message": j.message, "result": j.result, "error": j.error}

@router.delete("/jobs/{job_id}")
async def cancel_job(job_id: str):
    task = TASKS.get(job_id)
    if not task:
        raise HTTPException(404, "Unknown job")
    task.cancel()
    return {"cancelled": True}
```

WebSockets when you need two-way

PYTHON

```python
from fastapi import APIRouter, WebSocket, WebSocketDisconnect

router = APIRouter()

@router.websocket("/ws/chat")
async def ws_chat(ws: WebSocket):
    await ws.accept()
    history: list[dict] = []
    try:
        while True:
            msg = await ws.receive_json()
            if msg.get("type") == "cancel":
                await ws.send_json({"type": "cancelled"})
                continue

            history.append({"role": "user", "content": msg["text"]})
            await ws.send_json({"type": "status", "message": "thinking"})

            buf = []
            async for token in llm.astream_chat(history):
                buf.append(token)
                await ws.send_json({"type": "token", "text": token})
            history.append({"role": "assistant", "content": "".join(buf)})
            await ws.send_json({"type": "done"})
    except WebSocketDisconnect:
        pass                                    # normal; the user closed the tab
```

✦ **Prefer SSE unless you genuinely need upstream messages**

SSE is plain HTTP: it works through every proxy, reconnects automatically in browsers, and needs no special infrastructure. WebSockets need sticky sessions behind a load balancer, keep-alive tuning, and their own reconnection logic. For streaming an LLM answer, SSE is strictly simpler. Reach for WebSockets when the client must interrupt mid-generation or when you have a genuinely bidirectional flow.

Timeouts, retries and rate limits against the model provider

PYTHON

```
import asyncio, random, time
import httpx

class LLMTimeout(Exception): pass
class LLMRateLimited(Exception): pass

class LLMClient:
    def __init__(self, api_key: str, model: str, timeout_s: float = 60.0,
                 max_concurrency: int = 8):
        self.model = model
        self._sem = asyncio.Semaphore(max_concurrency)
        self._client = httpx.AsyncClient(
            base_url="https://api.openai.com/v1",
            headers={"Authorization": f"Bearer {api_key}"},
            timeout=httpx.Timeout(timeout_s, connect=10.0, read=timeout_s),
            limits=httpx.Limits(max_connections=max_concurrency * 2,
                                max_keepalive_connections=max_concurrency),
        )

    async def _post(self, path: str, payload: dict, attempts: int = 4) -> dict:
        async with self._sem:
            for attempt in range(1, attempts + 1):
                try:
                    r = await self._client.post(path, json=payload)
                except httpx.ReadTimeout:
                    if attempt == attempts:
                        raise LLMTimeout(f"model timed out after {attempts} attempts")
                    await asyncio.sleep(random.uniform(0, 2 ** attempt))
                    continue

                if r.status_code == 429:
                    wait = float(r.headers.get("retry-after", 2 ** attempt))
                    if attempt == attempts:
                        raise LLMRateLimited("rate limited; try again shortly")
                    await asyncio.sleep(wait + random.uniform(0, 1))
                    continue
                if r.status_code >= 500:
                    if attempt == attempts:
                        r.raise_for_status()
                    await asyncio.sleep(random.uniform(0, 2 ** attempt))
                    continue

                r.raise_for_status()
                return r.json()

    async def answer(self, question: str, chunks: list, temperature: float = 0.0):
        context = "\n\n---\n\n".join(c.text for c in chunks)
        data = await self._post("/chat/completions", {
```

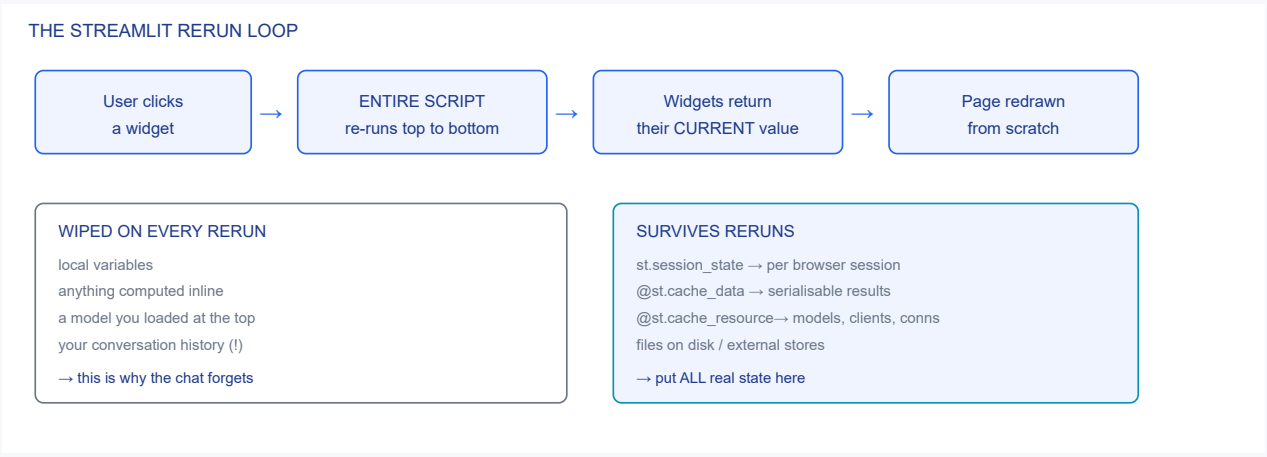
```
            "model": self.model,
            "temperature": temperature,
            "messages": [
                {"role": "system",
                 "content": "Answer ONLY from the provided context. "
                            "If the answer is not present, say so explicitly."},
                {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {questi
            ],
        })
        return data["choices"][0]["message"]["content"]
```

> ✕ **Unbounded concurrency is how you get your key suspended**
>
> A demo where someone pastes 200 questions into a loop can fire 200 simultaneous requests, hit the provider's rate limit, trigger aggressive retries, and in the worst case get the organisation's key throttled for everyone — including the production system of a different team at the customer. The semaphore above is four lines and prevents this entirely. Set it before the first demo, not after the incident.

Streamlit Deep Dive

Streamlit's entire design rests on one idea that is bizarre until it clicks: **your script re-runs from top to bottom on every single interaction**. There is no component tree, no virtual DOM, no event handlers in the usual sense. Once you accept that, everything else — session state, caching, forms, fragments — is a consequence, and you can build a convincing app in an afternoon.

The rerun model



Everything confusing about Streamlit follows from this single loop. Memorise the two boxes.

session_state

PYTHON

```python
import streamlit as st

# Initialise ONCE. This guard runs on the first script execution only;
# on subsequent reruns the key already exists and is left alone.
if "messages" not in st.session_state:
    st.session_state.messages = []
if "doc_count" not in st.session_state:
    st.session_state.doc_count = 0
if "last_error" not in st.session_state:
    st.session_state.last_error = None

# Widgets can bind directly to a state key
st.text_input("Your name", key="user_name")      # writes st.session_state.user_name

# Callbacks fire BEFORE the rerun, so state is already updated when the script runs
def clear_chat():
    st.session_state.messages = []
    st.session_state.last_error = None

st.button("Clear conversation", on_click=clear_chat)

# Two widgets can never share a key - this raises DuplicateWidgetID
# st.slider("A", key="k"); st.slider("B", key="k")   # error
```

The two session_state traps

1. Assigning to a widget-bound key after the widget is created raises `StreamlitAPIException`. Mutate it in an `on_click` callback instead, which runs before the rerun. **2. session_state is per browser session, in memory, in one process.** If the app restarts, or you scale to two replicas without sticky sessions, every user loses their state mid-demo. For a prototype that is acceptable; just never put anything in session_state you cannot afford to lose, and pin the deployment to one replica.

Caching: the two decorators and when each is wrong

	`@st.cache_data`	`@st.cache_resource`
Caches	A **copy** of the return value (pickled)	The **object itself**, shared globally
Use for	DataFrames, query results, API responses, computations	ML models, DB connections, HTTP clients, vector indexes
Mutation safety	Safe — each caller gets its own copy	Unsafe — all sessions share one object, so it must be thread-safe
Scope	Shared across all users of the process	Shared across all users of the process
Wrong choice symptom	Using it for a model: slow, or "cannot pickle" errors	Using it for a DataFrame: one user's edit changes everyone's data

PYTHON

```python
import pandas as pd
import streamlit as st

@st.cache_resource                        # one index for the whole process
def get_index():
    from app.services.retrieval import VectorIndex
    return VectorIndex.load("./data/index")

@st.cache_resource
def get_api_client():
    import httpx
    return httpx.Client(base_url=st.secrets["API_URL"], timeout=120.0)

@st.cache_data(ttl=600, show_spinner="Loading data ... ")
def load_orders(customer_id: str, as_of: str) -> pd.DataFrame:
    """Keyed on the ARGUMENTS. Different args = different cache entry."""
    return pd.read_parquet(f"data/orders_{customer_id}_{as_of}.parquet")

@st.cache_data(ttl=3600, max_entries=200)
def embed_query(text: str) -> list[float]:
    return get_api_client().post("/embed", json={"text": text}).json()["vector"]

# Arguments must be hashable. Prefix with an underscore to skip hashing a param.
@st.cache_data
def search(_index, query: str, k: int):      # _index is not hashed
    return _index.search(query, k=k)

# Manual invalidation - wire this to a "Refresh" button in the UI
if st.button("Refresh data"):
    load_orders.clear()                       # clear one function
    st.cache_data.clear()                     # clear everything
    st.rerun()
```

✦ **Cache the expensive thing, not the cheap thing**

The pattern that makes a Streamlit demo feel instant: `cache_resource` for anything loaded once (a 2-second model load becomes a one-time cost rather than a per-click cost) and `cache_data` with a TTL for anything fetched. Without caching, every keystroke in a text input re-loads your model. With it, the app feels like a native tool. This is usually the single biggest perceived-performance fix available to you.

Forms: batching inputs so the script does not rerun per keystroke

PYTHON

```python
with st.form("search_form", clear_on_submit=False, border=True):
    col1, col2 = st.columns([3, 1])
    with col1:
        query = st.text_input("Question", placeholder="What is our refund policy?")
    with col2:
        top_k = st.number_input("Sources", min_value=1, max_value=20, value=6)

    scope = st.multiselect("Scope", ["policies", "contracts", "manuals"], default=["po
    advanced = st.expander("Advanced")
    with advanced:
        temperature = st.slider("Temperature", 0.0, 1.0, 0.0, 0.1)
        rerank = st.checkbox("Re-rank results", value=True)

    submitted = st.form_submit_button("Ask", type="primary", use_container_width=True)

if submitted:
    if not query.strip():
        st.warning("Enter a question first.")
    else:
        with st.spinner("Searching …"):
            result = call_api(query, top_k, scope, temperature, rerank)
        st.session_state.messages.append({"role": "user", "content": query})
        st.session_state.messages.append({"role": "assistant", "content": result})
```

Layout

PYTHON

```python
st.set_page_config(
    page_title="Document Assistant",
    page_icon=":mag:",
    layout="wide",                      # "centered" is the default and often too nar
    initial_sidebar_state="expanded",
)

with st.sidebar:
    st.header("Settings")
    st.selectbox("Model", ["gpt-4o-mini", "gpt-4o"], key="model")
    st.slider("Sources to retrieve", 1, 20, 6, key="top_k")
    st.divider()
    st.caption(f"Index: {get_index().size:,} chunks")
    st.caption(f"Updated: {index_updated_at():%d %b %Y %H:%M}")

tab_ask, tab_docs, tab_eval = st.tabs(["Ask", "Documents", "Evaluation"])

with tab_ask:
    left, right = st.columns([2, 1], gap="large")
    with left:
        st.subheader("Conversation")
        # ... chat area
    with right:
        st.subheader("Sources")
        with st.container(height=520, border=True):
            for c in st.session_state.get("last_sources", []):
                with st.expander(f"{c['title']} (p.{c['page']}) - {c['score']:.2f}"):
                    st.markdown(c["snippet"])

with tab_docs:
    m1, m2, m3 = st.columns(3)
    m1.metric("Documents", 1_284, delta="+12 today")
    m2.metric("Chunks", 18_902)
    m3.metric("Last reindex", "2h ago")
    st.dataframe(documents_df(), use_container_width=True, hide_index=True)
```

Chat UI with streaming

PYTHON

```python
import streamlit as st

st.title("Policy Assistant")

if "messages" not in st.session_state:
    st.session_state.messages = []

# 1. Replay the whole history on every rerun - this is NOT wasteful,
#    it is how Streamlit works, and it is fast because it is just markdown.
for msg in st.session_state.messages:
    with st.chat_message(msg["role"], avatar="&#129302;" if msg["role"] == "assistant"
        st.markdown(msg["content"])
        if msg.get("sources"):
            with st.expander(f"{len(msg['sources'])} sources"):
                for s in msg["sources"]:
                    st.markdown(f"**{s['title']}** (p.{s['page']}) - _{s['score']:.2f}
                    st.caption(s["snippet"])

# 2. Input is pinned to the bottom of the page automatically
if prompt := st.chat_input("Ask about a policy ... "):
    st.session_state.messages.append({"role": "user", "content": prompt})
    with st.chat_message("user"):
        st.markdown(prompt)

    with st.chat_message("assistant"):
        try:
            # st.write_stream consumes any generator and returns the full text
            full = st.write_stream(stream_answer(prompt, api_url(), api_key()))
            sources = st.session_state.get("pending_sources", [])
            st.session_state.messages.append(
                {"role": "assistant", "content": full, "sources": sources})
        except Exception as e:
            st.error(f"Could not get an answer: {e}")
            st.session_state.messages.append(
                {"role": "assistant",
                 "content": "_Sorry - that request failed. Try again or rephrase._"})

# 3. Feedback, captured into your eval set. Do this from day one.
if st.session_state.messages and st.session_state.messages[-1]["role"] == "assistant":
    c1, c2, _ = st.columns([1, 1, 8])
    if c1.button(":thumbsup:", key=f"up_{len(st.session_state.messages)}"):
        log_feedback(st.session_state.messages[-2:], rating=1)
        st.toast("Thanks - logged.")
    if c2.button(":thumbsdown:", key=f"down_{len(st.session_state.messages)}"):
        log_feedback(st.session_state.messages[-2:], rating=-1)
        st.toast("Thanks - logged for review.")
```

File uploads and multipage apps

PYTHON

```python
uploaded = st.file_uploader(
    "Upload policy documents",
    type=["pdf", "docx", "txt", "md"],
    accept_multiple_files=True,
    help="Max 25 MB per file. Files are indexed locally and not sent anywhere else.",
)

if uploaded:
    progress = st.progress(0.0, text="Starting …")
    results = []
    for i, f in enumerate(uploaded, start=1):
        progress.progress(i / len(uploaded), text=f"Indexing {f.name} …")
        # UploadedFile is a BytesIO-like object; .getvalue() gives bytes
        results.append(index_bytes(f.name, f.getvalue()))
    progress.empty()

    ok = [r for r in results if r["status"] == "ok"]
    bad = [r for r in results if r["status"] ≠ "ok"]
    st.success(f"Indexed {len(ok)} file(s), {sum(r['chunks'] for r in ok)} chunks.")
    if bad:
        st.warning("Some files could not be processed:")
        st.dataframe(bad, hide_index=True)
```

PYTHON

```python
# Multipage: ui/app.py defines navigation explicitly (st.navigation API)
import streamlit as st

pages = {
    "Assistant": [
        st.Page("pages/ask.py",      title="Ask",        icon=":material/search:", def
        st.Page("pages/documents.py",title="Documents",  icon=":material/folder:"),
    ],
    "Admin": [
        st.Page("pages/evaluation.py", title="Evaluation", icon=":material/science:"),
        st.Page("pages/settings.py",   title="Settings",   icon=":material/settings:")
    ],
}
nav = st.navigation(pages)
st.set_page_config(page_title="Document Assistant", layout="wide")
nav.run()

# The older convention still works: a pages/ directory with 1_Ask.py, 2_Documents.py
# where the number controls order and the name becomes the label.
```

Fragments: rerun part of the page

PYTHON

```python
# Without a fragment, this clock would rerun the ENTIRE script every 5 seconds,
# re-calling your API and resetting scroll position. A fragment reruns only itself.
@st.fragment(run_every="5s")
def live_status():
    status = get_api_client().get("/health").json()
    c1, c2, c3 = st.columns(3)
    c1.metric("API", "up" if status["ok"] else "down")
    c2.metric("Queue depth", status["queue"])
    c3.metric("p95 latency", f"{status['p95_ms']} ms")

live_status()

@st.fragment
def source_panel():
    """Reruns only when ITS widgets change - the chat above is untouched."""
    filt = st.text_input("Filter sources")
    for s in st.session_state.get("last_sources", []):
        if filt.lower() in s["title"].lower():
            st.caption(f"{s['title']} - {s['score']:.2f}")
```

Streamlit anti-patterns

Anti-pattern	Symptom	Fix
Loading a model at module level without caching	App takes 8 s per click	`@st.cache_resource`
Business logic inside the Streamlit script	Untestable; cannot be reused	Move to `services/`; the UI only calls and displays
Relying on session_state for anything durable	Lost on restart or scale-out	Write to a file, SQLite or the backend
Long synchronous work with no feedback	Looks frozen; users click again	`st.spinner`, `st.status`, `st.progress`
Mutating a `cache_data` result in place	Confusing stale values	Copy before mutating, or use `cache_resource` deliberately
Deep `if/else` trees to simulate pages	Unmaintainable after ~200 lines	`st.navigation` / `pages/`
Secrets in the script	Leaked on screen-share	`st.secrets` / env vars

Gradio Deep Dive

Gradio was built for one job — put a UI in front of a model and share it — and it is still the fastest way to do exactly that. Where Streamlit gives you a general-purpose app framework, Gradio gives you a model demo with input and output components, an event graph, a built-in queue, and a public link in one line. On a customer site, its killer feature is often that one line.

Interface: the five-line demo

PYTHON

```
import gradio as gr

def classify(text: str, threshold: float) -> dict:
    scores = model.predict_proba([text])[0]
    return {label: float(s) for label, s in zip(model.classes_, scores) if s ≥ thresh

demo = gr.Interface(
    fn=classify,
    inputs=[
        gr.Textbox(lines=4, label="Support ticket text",
                   placeholder="Paste the customer's message … "),
        gr.Slider(0.0, 1.0, value=0.1, step=0.05, label="Minimum confidence"),
    ],
    outputs=gr.Label(num_top_classes=5, label="Predicted category"),
    title="Ticket Classifier",
    description="Classifies incoming support tickets. Trained on 42k historical ticket
    examples=[
        ["My invoice shows the wrong VAT rate", 0.1],
        ["Cannot log in after the password reset", 0.1],
        ["Requesting a refund for order 88213", 0.1],
    ],
    cache_examples=True,            # precomputes example outputs — instant on load
    flagging_mode="manual",         # gives users a flag button; writes to a CSV
)

if __name__ == "__main__":
    demo.launch()
```

Blocks: when you need real layout and event wiring

PYTHON

```python
import gradio as gr

with gr.Blocks(title="Document Assistant", theme=gr.themes.Soft()) as demo:
    gr.Markdown("# Document Assistant\nAsk questions over the indexed policy corpus.")

    # State is explicit in Gradio - it is a component you pass around
    session = gr.State({"history": [], "sources": []})

    with gr.Row():
        with gr.Column(scale=3):
            chat   = gr.Chatbot(height=460, type="messages", show_copy_button=True)
            with gr.Row():
                box  = gr.Textbox(placeholder="Ask a question ... ", scale=8,
                                  show_label=False, container=False)
                send = gr.Button("Send", variant="primary", scale=1)
                stop = gr.Button("Stop", variant="stop", scale=1)
        with gr.Column(scale=1):
            gr.Markdown("### Settings")
            model_dd = gr.Dropdown(["gpt-4o-mini", "gpt-4o"], value="gpt-4o-mini", lab
            topk     = gr.Slider(1, 20, value=6, step=1, label="Sources")
            gr.Markdown("### Retrieved sources")
            sources  = gr.JSON(label=None)

    with gr.Accordion("Upload documents", open=False):
        files  = gr.File(file_count="multiple", file_types=[".pdf", ".txt", ".md"])
        upload = gr.Button("Index")
        status = gr.Markdown()

    clear = gr.Button("Clear conversation")

    def respond(message, history, model, k, state):
        chunks = index.search(message, k=k)
        state["sources"] = [c.brief() for c in chunks]
        history = history + [{"role": "user", "content": message}]
        partial = ""
        for token in llm.stream(message, chunks, model=model):
            partial += token
            yield history + [{"role": "assistant", "content": partial}], state["source

    submit_event = send.click(
        respond,
        inputs=[box, chat, model_dd, topk, session],
        outputs=[chat, sources, session],
    ).then(lambda: "", outputs=box)          # clear the input after sending

    box.submit(respond, [box, chat, model_dd, topk, session], [chat, sources, session]
    stop.click(None, cancels=[submit_event]) # genuine cancellation of a running gener
    clear.click(lambda: ([], [], {"history": [], "sources": []}), outputs=[chat, sourc
```

```
    upload.click(index_files, inputs=files, outputs=status)

demo.queue(default_concurrency_limit=4, max_size=32)
demo.launch(server_name="0.0.0.0", server_port=7860)
```

ChatInterface: the shortest path to a chat demo

PYTHON

```
import gradio as gr

def chat_fn(message, history):
    """history is a list of {'role','content'} dicts when type='messages'."""
    context = retrieve(message, k=6)
    partial = ""
    for token in llm.stream(message, context, history=history):
        partial += token
        yield partial                       # yielding makes it stream, no extra confi

demo = gr.ChatInterface(
    fn=chat_fn,
    type="messages",
    title="Policy Assistant",
    description="Answers come only from indexed policy documents.",
    examples=["What is the notice period?", "How do I claim travel expenses?"],
    cache_examples=False,
    chatbot=gr.Chatbot(height=520, show_copy_button=True),
    textbox=gr.Textbox(placeholder="Ask about a policy …", container=False, scale=7),
    theme=gr.themes.Soft(),
)

demo.queue(default_concurrency_limit=4).launch(
    auth=("demo", "the-password-you-told-them"),   # minimal gate, see auth section
    share=False,
)
```

Queueing and concurrency

PYTHON

```python
# Gradio's queue is the thing that stops a demo collapsing under three users.
demo.queue(
    default_concurrency_limit=4,     # how many of ANY event run at once
    max_size=50,                     # beyond this, users are told the queue is full
    status_update_rate="auto",
)

# Per-event overrides: embedding is GPU bound, retrieval is not
with gr.Blocks() as demo:
    btn.click(expensive_gpu_fn, inputs, outputs, concurrency_limit=1,
              concurrency_id="gpu")        # named lane shared with other GPU events
    btn2.click(cheap_lookup, inputs, outputs, concurrency_limit=20)
```

Sharing links

PYTHON

```python
demo.launch(share=True)
# Creates a *.gradio.live tunnel, valid for ~1 week, reachable from anywhere.
```

Never use `share=True` with customer data

That URL is a public tunnel through Gradio's infrastructure to a process on your laptop. There is no authentication unless you add `auth=`, the link is guessable enough that scanners find these, and your laptop is now an internet-facing server inside the customer's network. It is brilliant for sharing a toy model with a colleague and completely unacceptable for a prototype containing a customer's contracts. If you need remote access, deploy it properly behind their SSO — that is a half-day of work and it is the difference between a professional engagement and an incident report.

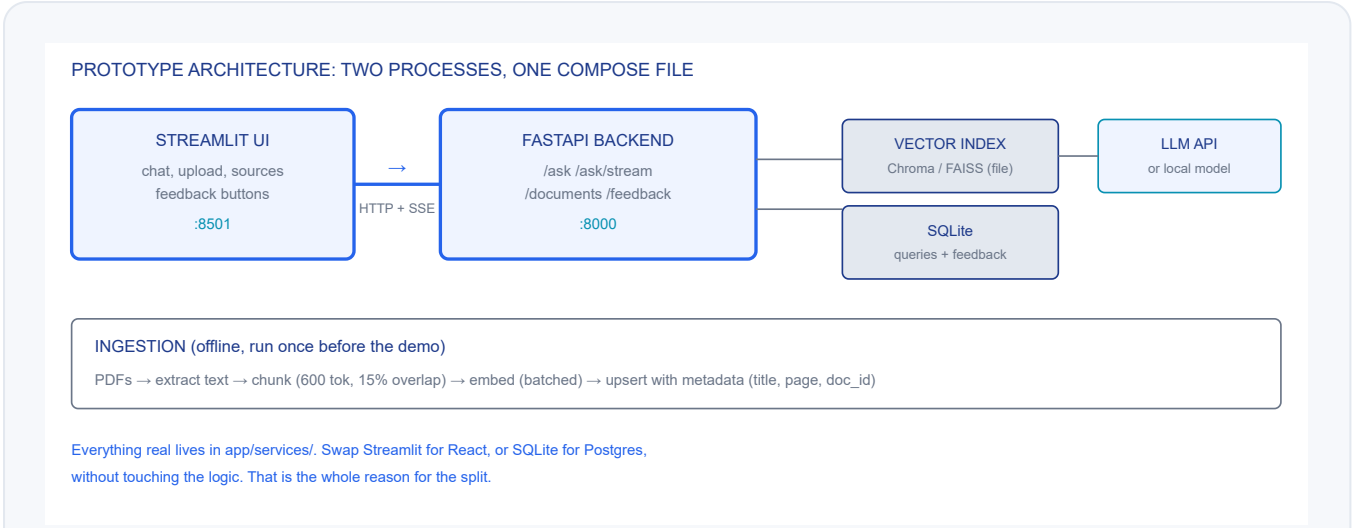
Streamlit or Gradio?

Choose Gradio when	Choose Streamlit when
The app is fundamentally "input → model → output"	The app has multiple views, tables, filters, dashboards
You want a chat UI in under ten lines	You need rich data display — dataframes, charts, metrics, editors
You are deploying to Hugging Face Spaces	You are deploying to the customer's cloud
You need a temporary share link for a colleague	You need multipage navigation and a sidebar of controls
You need built-in queueing and cancellation	You want the script-rerun simplicity
You want the UI to double as a callable API	You will call a separate FastAPI backend anyway
Image/audio/video input and output components	Tabular and textual workflows

> **In practice**
> For FDE work with enterprise customers, Streamlit wins most of the time, because the demos are usually "chat plus a table plus some filters plus an upload" and because it looks more like an internal business tool. Gradio wins for pure model demos, for anything with images or audio, and when you need a shareable link to a model within minutes. Both are prototyping tools; neither is where you should land in production.

Worked Example: Document Q&A Over a Customer's PDFs

This is the prototype you will build more than any other. A customer has a folder of policy documents, contracts or manuals, and someone spends hours a week answering questions that are answered somewhere in those files. Here is the whole thing — FastAPI backend, Streamlit frontend — small enough to read and real enough to demo.



Two containers, no cluster, no queue. Deployable to a laptop, a customer VM or a container service unchanged.

Backend: the retrieval service

PYTHON

```python
# app/services/retrieval.py - NO FastAPI imports. Pure logic, fully testable.
from __future__ import annotations

import hashlib
import re
from dataclasses import dataclass
from pathlib import Path

import chromadb
from chromadb.utils import embedding_functions


@dataclass(frozen=True)
class Chunk:
    chunk_id: str
    document_id: str
    title: str
    page: int | None
    text: str
    score: float = 0.0

    def brief(self) -> dict:
        return {"document_id": self.document_id, "title": self.title, "page": self.pag
                "score": round(self.score, 3),
                "snippet": self.text[:280] + (" ..." if len(self.text) > 280 else "")}


def chunk_text(text: str, target_tokens: int = 600, overlap_ratio: float = 0.15) -> li
    """Paragraph-aware chunking. Roughly 4 chars per token for English.
    Splitting on paragraph boundaries matters far more than the exact size:
    a chunk that ends mid-sentence retrieves badly and reads badly in a citation."""
    target_chars = target_tokens * 4
    overlap = int(target_chars * overlap_ratio)

    paras = [p.strip() for p in re.split(r"\n\s*\n", text) if p.strip()]
    chunks, current = [], ""
    for p in paras:
        if len(current) + len(p) + 2 ≤ target_chars:
            current = f"{current}\n\n{p}" if current else p
        else:
            if current:
                chunks.append(current)
            # Carry the tail of the previous chunk forward so a fact that
            # straddles a boundary is retrievable from either side.
            current = (current[-overlap:] + "\n\n" + p) if current and overlap else p
    if current:
        chunks.append(current)
    return chunks
```

```python
class VectorIndex:
    def __init__(self, client, collection):
        self._client, self._col = client, collection

    @classmethod
    def load(cls, path: str, embedding_model: str = "all-MiniLM-L6-v2") -> "VectorInde
        Path(path).mkdir(parents=True, exist_ok=True)
        client = chromadb.PersistentClient(path=path)
        ef = embedding_functions.SentenceTransformerEmbeddingFunction(model_name=embed
        col = client.get_or_create_collection(
            name="documents", embedding_function=ef,
            metadata={"hnsw:space": "cosine"},
        )
        return cls(client, col)

    @property
    def size(self) -> int:
        return self._col.count()

    def add_document(self, document_id: str, title: str, pages: list[str]) -> int:
        """Idempotent: deletes existing chunks for this doc before inserting."""
        self._col.delete(where={"document_id": document_id})

        ids, docs, metas = [], [], []
        for page_no, page_text in enumerate(pages, start=1):
            for i, chunk in enumerate(chunk_text(page_text)):
                cid = hashlib.sha256(f"{document_id}|{page_no}|{i}".encode()).hexdiges
                ids.append(cid)
                docs.append(chunk)
                metas.append({"document_id": document_id, "title": title, "page": page

        # Chroma has a per-call batch limit; chunk the upsert.
        for s in range(0, len(ids), 256):
            self._col.upsert(ids=ids[s:s+256], documents=docs[s:s+256], metadatas=meta
        return len(ids)

    def search(self, query: str, k: int = 6, filters: dict | None = None) -> list[Chun
        res = self._col.query(query_texts=[query], n_results=k, where=filters or None)
        if not res["ids"] or not res["ids"][0]:
            return []
        out = []
        for cid, doc, meta, dist in zip(res["ids"][0], res["documents"][0],
                                        res["metadatas"][0], res["distances"][0]):
            out.append(Chunk(chunk_id=cid, document_id=meta["document_id"],
                             title=meta["title"], page=meta.get("page"),
                             text=doc, score=1.0 - dist))   # cosine distance → simil
        return out
```

```
    def documents(self) -> list[dict]:
        all_meta = self._col.get(include=["metadatas"])["metadatas"]
        seen: dict[str, dict] = {}
        for m in all_meta:
            d = seen.setdefault(m["document_id"], {"document_id": m["document_id"],
                                                   "title": m["title"], "chunks": 0})
            d["chunks"] += 1
        return sorted(seen.values(), key=lambda d: d["title"])

    def close(self) -> None:
        pass
```

Backend: the answering service

PYTHON

```python
# app/services/llm.py
from __future__ import annotations

import json
from collections.abc import AsyncIterator
from dataclasses import dataclass

import httpx

from app.services.retrieval import Chunk

SYSTEM_PROMPT = """You answer questions using ONLY the numbered context passages provi

Rules:
- Cite the passage number in square brackets for every factual claim, e.g. [2].
- If the context does not contain the answer, reply exactly:
  "I could not find that in the indexed documents."
  Do not use outside knowledge to fill the gap.
- Be concise. Three sentences unless the question needs more.
- If passages disagree, say so and cite both."""


@dataclass
class Answer:
    text: str
    citations: list[dict]
    truncated: bool = False


def build_context(chunks: list[Chunk], max_chars: int = 12_000) -> tuple[str, bool]:
    parts, used, truncated = [], 0, False
    for i, c in enumerate(chunks, start=1):
        block = f"[{i}] {c.title}" + (f" (page {c.page})" if c.page else "") + f"\n{c.
        if used + len(block) > max_chars:
            truncated = True
            break
        parts.append(block)
        used += len(block)
    return "\n\n---\n\n".join(parts), truncated


class LLMClient:
    def __init__(self, api_key: str, model: str, timeout_s: float = 60.0, base_url: st
        self.model = model
        self._client = httpx.AsyncClient(
            base_url=base_url or "https://api.openai.com/v1",
            headers={"Authorization": f"Bearer {api_key}"},
            timeout=httpx.Timeout(timeout_s, connect=10.0),
```

```
        )

    async def answer(self, question: str, chunks: list[Chunk], temperature: float = 0.
        context, truncated = build_context(chunks)
        r = await self._client.post("/chat/completions", json={
            "model": self.model,
            "temperature": temperature,
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": f"Context:\n\n{context}\n\nQuestion: {ques
            ],
        })
        r.raise_for_status()
        text = r.json()["choices"][0]["message"]["content"]
        return Answer(text=text, citations=[c.brief() for c in chunks], truncated=trun

    async def astream(self, question: str, chunks: list[Chunk],
                      temperature: float = 0.0) -> AsyncIterator[str]:
        context, _ = build_context(chunks)
        payload = {
            "model": self.model, "temperature": temperature, "stream": True,
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": f"Context:\n\n{context}\n\nQuestion: {ques
            ],
        }
        async with self._client.stream("POST", "/chat/completions", json=payload) as r
            r.raise_for_status()
            async for line in r.aiter_lines():
                if not line.startswith("data: "):
                    continue
                body = line[6:]
                if body == "[DONE]":
                    return
                delta = json.loads(body)["choices"][0]["delta"]
                if "content" in delta:
                    yield delta["content"]
```

Backend: routes

PYTHON

```python
# app/routers/ask.py
import json, time, uuid
from typing import Annotated

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import StreamingResponse

from app.deps import get_index, get_llm
from app.models.schemas import AskRequest, AskResponse
from app.services.feedback import log_query, log_feedback
from app.services.llm import LLMClient
from app.services.retrieval import VectorIndex

router = APIRouter(tags=["ask"])


@router.post("/ask", response_model=AskResponse)
async def ask(req: AskRequest,
              index: Annotated[VectorIndex, Depends(get_index)],
              llm: Annotated[LLMClient, Depends(get_llm)]) -> AskResponse:
    t0, rid = time.perf_counter(), str(uuid.uuid4())[:8]
    chunks = index.search(req.question, k=req.top_k)
    if not chunks:
        raise HTTPException(404, "No relevant passages found.")
    result = await llm.answer(req.question, chunks, temperature=req.temperature)
    latency = int((time.perf_counter() - t0) * 1000)

    # Log EVERY query. This becomes your evaluation set and your demo transcript.
    log_query(rid, req.question, result.text, [c.chunk_id for c in chunks], latency)

    return AskResponse(answer=result.text, citations=result.citations,
                       model=llm.model, latency_ms=latency,
                       request_id=rid, truncated=result.truncated)


@router.post("/ask/stream")
async def ask_stream(req: AskRequest,
                     index: Annotated[VectorIndex, Depends(get_index)],
                     llm: Annotated[LLMClient, Depends(get_llm)]):
    async def events():
        rid = str(uuid.uuid4())[:8]
        t0 = time.perf_counter()
        yield f"data: {json.dumps({'type':'status','message':'Searching documents'})}\

        chunks = index.search(req.question, k=req.top_k)
        if not chunks:
            yield f"data: {json.dumps({'type':'error','message':'No relevant passages
            return
```

```
        yield f"data: {json.dumps({'type':'sources','sources':[c.brief() for c in chun

        buf = []
        try:
            async for tok in llm.astream(req.question, chunks, temperature=req.tempera
                buf.append(tok)
                yield f"data: {json.dumps({'type':'token','text':tok})}\n\n"
        except Exception as e:
            yield f"data: {json.dumps({'type':'error','message':str(e)})}\n\n"
            return

        latency = int((time.perf_counter() - t0) * 1000)
        log_query(rid, req.question, "".join(buf), [c.chunk_id for c in chunks], laten
        yield f"data: {json.dumps({'type':'done','request_id':rid,'latency_ms':latency

    return StreamingResponse(events(), media_type="text/event-stream",
                             headers={"Cache-Control": "no-cache", "X-Accel-Buffering"


@router.post("/feedback")
async def feedback(request_id: str, rating: int, comment: str | None = None):
    if rating not in (-1, 1):
        raise HTTPException(422, "rating must be -1 or 1")
    log_feedback(request_id, rating, comment)
    return {"ok": True}
```

PYTHON

```python
# app/services/feedback.py - SQLite is enough, and it is one file you can hand over.
import sqlite3, json, datetime
from pathlib import Path

DB = Path("data/app.db")

def _conn():
    DB.parent.mkdir(parents=True, exist_ok=True)
    c = sqlite3.connect(DB, check_same_thread=False)
    c.execute("""create table if not exists queries (
        request_id text primary key, ts text, question text, answer text,
        chunk_ids text, latency_ms integer)""")
    c.execute("""create table if not exists feedback (
        request_id text, ts text, rating integer, comment text)""")
    return c

def log_query(request_id, question, answer, chunk_ids, latency_ms):
    with _conn() as c:
        c.execute("insert or replace into queries values (?,?,?,?,?,?)",
                  (request_id, datetime.datetime.now(datetime.UTC).isoformat(),
                   question, answer, json.dumps(chunk_ids), latency_ms))

def log_feedback(request_id, rating, comment=None):
    with _conn() as c:
        c.execute("insert into feedback values (?,?,?,?)",
                  (request_id, datetime.datetime.now(datetime.UTC).isoformat(), rating

def export_eval_set(path="data/eval_set.jsonl"):
    """Everything the customer asked during the demo, ready to become test cases."""
    with _conn() as c:
        rows = c.execute("""
            select q.question, q.answer, q.chunk_ids, f.rating
            from queries q left join feedback f using (request_id)
            order by q.ts""").fetchall()
    with open(path, "w", encoding="utf-8") as fh:
        for question, answer, chunk_ids, rating in rows:
            fh.write(json.dumps({"question": question, "answer": answer,
                                 "retrieved": json.loads(chunk_ids),
                                 "rating": rating}) + "\n")
    return len(rows)
```

Frontend: the Streamlit client

PYTHON

```python
# ui/api_client.py — the ONLY module that knows the backend exists
import json
import httpx
import streamlit as st


def _base() -> str:
    return st.secrets.get("API_URL", "http://localhost:8000")


@st.cache_resource
def client() -> httpx.Client:
    return httpx.Client(base_url=_base(), timeout=httpx.Timeout(300.0, connect=10.0),
                        headers={"X-API-Key": st.secrets.get("API_KEY", "dev")})


def health() -> dict:
    try:
        r = client().get("/health", timeout=5.0)
        return r.json() if r.status_code == 200 else {"ok": False}
    except Exception as e:
        return {"ok": False, "error": str(e)}


def documents() -> list[dict]:
    return client().get("/api/v1/documents").json()


def upload(files) -> dict:
    payload = [("files", (f.name, f.getvalue(), f.type)) for f in files]
    return client().post("/api/v1/documents", files=payload).json()


def ask_stream(question: str, top_k: int = 6):
    """Yields text tokens; stashes sources in session_state as they arrive."""
    with client().stream("POST", "/api/v1/ask/stream",
                         json={"question": question, "top_k": top_k}) as r:
        r.raise_for_status()
        for line in r.iter_lines():
            if not line or not line.startswith("data: "):
                continue
            ev = json.loads(line[6:])
            if ev["type"] == "sources":
                st.session_state.pending_sources = ev["sources"]
            elif ev["type"] == "token":
                yield ev["text"]
            elif ev["type"] == "error":
                raise RuntimeError(ev["message"])
```

```
            elif ev["type"] == "done":
                st.session_state.last_request_id = ev["request_id"]
                st.session_state.last_latency_ms = ev["latency_ms"]
                return


def send_feedback(request_id: str, rating: int, comment: str | None = None):
    client().post("/api/v1/feedback",
                  params={"request_id": request_id, "rating": rating},
                  json={"comment": comment})
```

PYTHON

```python
# ui/app.py
import streamlit as st

from api_client import ask_stream, documents, health, send_feedback, upload

st.set_page_config(page_title="Policy Assistant", page_icon=":books:", layout="wide")

for key, default in [("messages", []), ("pending_sources", []),
                     ("last_request_id", None), ("last_latency_ms", None)]:
    st.session_state.setdefault(key, default)

# --- Sidebar: status, controls, and the reset button you WILL need mid-demo ---
with st.sidebar:
    st.title("Policy Assistant")
    h = health()
    if h.get("ok"):
        st.success(f"Connected - {h.get('chunks', 0):,} chunks indexed")
    else:
        st.error("Backend unavailable")
        st.caption(h.get("error", ""))

    top_k = st.slider("Passages to retrieve", 1, 15, 6)
    st.divider()

    with st.expander("Add documents"):
        files = st.file_uploader("PDF, DOCX, TXT", type=["pdf", "docx", "txt", "md"],
                                 accept_multiple_files=True, label_visibility="collaps
        if files and st.button("Index files", use_container_width=True):
            with st.spinner("Indexing ..."):
                res = upload(files)
            st.success(f"Indexed {len(res['accepted'])} file(s)")
            if res["rejected"]:
                st.warning(res["rejected"])

    st.divider()
    if st.button("Reset conversation", use_container_width=True):
        st.session_state.messages = []
        st.session_state.pending_sources = []
        st.rerun()

    with st.expander("Indexed documents"):
        st.dataframe(documents(), hide_index=True, use_container_width=True)

# --- Main: chat on the left, citations on the right ---
chat_col, src_col = st.columns([2, 1], gap="large")

with chat_col:
    for i, m in enumerate(st.session_state.messages):
```

```
        with st.chat_message(m["role"]):
            st.markdown(m["content"])
            if m.get("latency_ms"):
                st.caption(f"{m['latency_ms']} ms")

    if prompt := st.chat_input("Ask about a policy ... "):
        st.session_state.messages.append({"role": "user", "content": prompt})
        with st.chat_message("user"):
            st.markdown(prompt)

        with st.chat_message("assistant"):
            try:
                text = st.write_stream(ask_stream(prompt, top_k=top_k))
                st.session_state.messages.append({
                    "role": "assistant", "content": text,
                    "sources": st.session_state.pending_sources,
                    "latency_ms": st.session_state.last_latency_ms,
                })
            except Exception as e:
                st.error("That request failed. Try rephrasing, or check the backend st
                st.caption(str(e))

    # Feedback on the most recent answer → straight into the eval set
    if st.session_state.messages and st.session_state.messages[-1]["role"] == "assista
        c1, c2, _ = st.columns([1, 1, 6])
        rid = st.session_state.last_request_id
        if c1.button("Helpful", use_container_width=True):
            send_feedback(rid, 1); st.toast("Thanks - logged.")
        if c2.button("Not helpful", use_container_width=True):
            send_feedback(rid, -1); st.toast("Logged for review.")

with src_col:
    st.subheader("Sources")
    sources = (st.session_state.messages[-1].get("sources")
               if st.session_state.messages else []) or st.session_state.pending_sourc
    if not sources:
        st.caption("Ask a question to see the passages used.")
    for i, s in enumerate(sources, start=1):
        with st.expander(f"[{i}] {s['title']}" + (f" - p.{s['page']}" if s['page'] els
                         expanded=(i == 1)):
            st.caption(f"Relevance {s['score']:.2f}")
            st.markdown(s["snippet"])
```

The ingestion script

PYTHON

```python
# scripts/ingest_pdfs.py - run this BEFORE the demo, not during it.
import sys, time
from pathlib import Path

import pypdf

from app.services.retrieval import VectorIndex


def extract_pages(path: Path) -> list[str]:
    reader = pypdf.PdfReader(str(path))
    pages = [(p.extract_text() or "").strip() for p in reader.pages]
    text_chars = sum(len(p) for p in pages)
    if text_chars < 200 * len(pages):
        # A scanned PDF. You will meet these at every customer.
        print(f"  WARNING: {path.name} looks scanned ({text_chars} chars across "
              f"{len(pages)} pages). OCR needed - skipping for now.")
    return pages


def main(folder: str, index_path: str = "./data/index"):
    index = VectorIndex.load(index_path)
    pdfs = sorted(Path(folder).rglob("*.pdf"))
    print(f"Found {len(pdfs)} PDFs")

    total_chunks, skipped, t0 = 0, 0, time.perf_counter()
    for i, path in enumerate(pdfs, start=1):
        try:
            pages = extract_pages(path)
            if not any(pages):
                skipped += 1
                continue
            n = index.add_document(document_id=path.stem, title=path.stem.replace("_",
                                   pages=pages)
            total_chunks += n
            print(f"[{i}/{len(pdfs)}] {path.name}: {len(pages)} pages, {n} chunks")
        except Exception as e:
            skipped += 1
            print(f"[{i}/{len(pdfs)}] FAILED {path.name}: {e}")

    print(f"\nDone in {time.perf_counter() - t0:.1f}s. "
          f"{total_chunks:,} chunks, {skipped} files skipped. Index size: {index.size:


if __name__ == "__main__":
    main(sys.argv[1] if len(sys.argv) > 1 else "./data/pdfs")
```

⚠ **Scanned PDFs are the most common surprise**

You will be given "the policy documents" and discover that a third of them are scans with no text layer. Check this on day one with the character-count heuristic above. Your options are OCR (Tesseract locally, or a cloud document-intelligence service if data residency allows), asking the customer for the original files, or scoping those documents out of the demo and saying so. All three are fine; discovering it the night before the demo is not.

Making a Prototype Demo-Proof

The prototype working on your machine is roughly half the job. The other half is making sure that nothing visible goes wrong in the forty minutes when it matters — on their network, on a projector, with the CTO watching, after a flight. Demo failures are not bad luck; they are a small set of predictable causes, and all of them are preventable.

The failure modes, ranked by how often they actually happen

#	Failure	Prevention
1	Customer guest wifi blocks your API calls or is unusably slow	Offline/demo mode with cached responses; mobile hotspot as backup; a recorded video as the last resort
2	Model provider is slow or rate-limits mid-demo	Cache every demo question's answer; short timeout with a cached fallback; a second provider configured
3	Someone types something you never tested and it 500s	Catch-all error handler that shows a friendly message, never a stack trace
4	Laptop sleeps / screensaver / notifications	Disable sleep and notifications; do not present from your personal machine's main profile
5	App state is dirty from the practice run	A visible Reset button and a one-command reseed
6	Fonts/layout break on the projector resolution	Test at 1280x720 and at 150% browser zoom
7	Cold start: first request takes 30 s while a model loads	Warm up in `lifespan`; fire a dummy request before they walk in
8	You demo on data that embarrasses someone in the room	Review every seeded record; no real salaries, no named individuals, no competitor names

Demo mode

PYTHON

```python
# app/services/demo.py
import hashlib, json
from pathlib import Path

CACHE = Path("data/demo_cache.json")

class DemoCache:
    """Answers canned responses for known questions. Two purposes:
    1) the demo works with no network at all
    2) known questions answer in 50 ms instead of 6 s, which looks far better"""

    def __init__(self, enabled: bool):
        self.enabled = enabled
        self._data = json.loads(CACHE.read_text()) if CACHE.exists() else {}

    @staticmethod
    def _key(question: str) -> str:
        norm = " ".join(question.lower().split()).rstrip("?.")
        return hashlib.sha256(norm.encode()).hexdigest()[:16]

    def get(self, question: str) -> dict | None:
        return self._data.get(self._key(question)) if self.enabled else None

    def put(self, question: str, answer: str, citations: list[dict]) -> None:
        self._data[self._key(question)] = {"answer": answer, "citations": citations}
        CACHE.write_text(json.dumps(self._data, indent=2))


# In the route:
#   cached = demo_cache.get(req.question)
#   if cached:
#       return AskResponse(**cached, model="cache", latency_ms=12, request_id="demo")
#
# Then, the day before the demo, run every planned question once with
# DEMO_MODE=false to populate the cache, and flip DEMO_MODE=true to present.
```

PYTHON

```python
# Graceful degradation: try live, fall back to cache, never show a stack trace.
async def answer_with_fallback(question: str, chunks, llm, cache) -> tuple[str, str]:
    try:
        result = await asyncio.wait_for(llm.answer(question, chunks), timeout=12.0)
        cache.put(question, result.text, result.citations)      # warm the cache as yo
        return result.text, "live"
    except (asyncio.TimeoutError, httpx.HTTPError, httpx.ConnectError):
        cached = cache.get(question)
        if cached:
            return cached["answer"], "cache"
        return ("I could not reach the model just now. The passages below are the "
                "most relevant ones from the documents."), "degraded"
```

The cache-as-you-go trick

Run through your entire demo script once on a good connection the day before. Every answer lands in the cache. On demo day, even if the network dies completely, every question you planned to ask returns instantly, and anything improvised still tries live. You get resilience and a visibly snappier demo for ten lines of code.

Seeded data and the reset button

PYTHON

```python
# scripts/seed.py - idempotent. Run it before every demo.
import shutil, sqlite3
from pathlib import Path

from app.services.retrieval import VectorIndex

def reset():
    for p in ["data/index", "data/app.db", "data/uploads"]:
        path = Path(p)
        if path.is_dir():
            shutil.rmtree(path)
        elif path.exists():
            path.unlink()

def seed():
    reset()
    index = VectorIndex.load("data/index")
    for doc in Path("data/samples").glob("*.txt"):
        index.add_document(doc.stem, doc.stem.replace("_", " "), [doc.read_text()])
    print(f"Seeded {index.size} chunks from data/samples/")

if __name__ == "__main__":
    seed()
```

Error states a customer can read

PYTHON

```python
ERROR_MESSAGES = {
    "no_results":   ("No matching passages", "Nothing in the indexed documents matched
                                             "Try different wording, or check the scop
    "timeout":      ("That took too long",   "The model did not respond in time. Try a
    "rate_limited": ("Busy right now",       "We are being rate limited. Wait a few se
    "upstream":     ("Service unavailable",  "The language model service is not reacha
                                             "Retrieval still works - see the passages
    "too_large":    ("Too much context",     "That question matched too much text. Red
}

def show_error(code: str, detail: str | None = None):
    title, body = ERROR_MESSAGES.get(code, ("Something went wrong", "Please try again.
    st.error(f"**{title}**  \n{body}")
    if detail:
        with st.expander("Technical detail"):
            st.code(detail)             # available, but not shoved in the customer's
```

The pre-demo checklist

- [ ] Full run-through on the **actual machine and network** you will present from, at least two hours before.
- [ ] `python scripts/seed.py` run; data is clean, realistic and contains nothing embarrassing.
- [ ] Demo cache populated for every question in your script.
- [ ] Backend warm — one dummy request fired so no model loading happens live.
- [ ] Browser at the resolution you will project; zoom set; only the demo tab open.
- [ ] Notifications off, sleep disabled, Do Not Disturb on, phone silenced.
- [ ] Mobile hotspot tethered and tested as a fallback.
- [ ] A screen recording of the full happy path saved locally, ready to play if everything fails.
- [ ] Reset button tested — you will need it between sections.
- [ ] Three improvised questions tried, so you know the error path looks acceptable.
- [ ] Decided in advance what you will say when something breaks: "that's the prototype's error path doing its job — let me reset" is a perfectly good line.
- [ ] Someone else on the call ready to drive the conversation while you fix anything.

✓ **How to handle a live failure**
Do not apologise repeatedly, do not start debugging on the shared screen, and never show a terminal full of red. Say what happened in one sentence, hit reset, and move on. If it fails twice, switch to the recording and say "I'll show you the recorded version and we'll debug offline — the interesting part is what it does, not whether the demo wifi holds." Customers forgive technical failure handled calmly; they remember panic.

Lightweight Auth for Prototypes

A prototype containing a customer's documents on a public URL with no authentication is a security incident, not a demo. The good news is that every option here takes under an

hour, and picking one is almost always enough to get past a security conversation that would otherwise block you for weeks.

The options, from weakest to strongest

Option	Effort	Good enough for	Not good enough for
Nothing, localhost only	0	Running on your laptop, screen-shared	Anything anyone else can reach
Shared password (Gradio `auth=`, Streamlit gate)	10 min	A handful of named people, non-sensitive data	Real PII, many users, audit requirements
HTTP Basic + HTTPS	20 min	Internal demos behind a VPN	Anything where you need to know *who* did what
API key per user	30 min	Machine-to-machine; attributing usage	Browser-facing UIs
Signed, expiring links	30 min	Sharing a single result or report externally	An ongoing interactive app
OAuth proxy (oauth2-proxy, Easy Auth, IAP)	1-2 h	Anything inside the customer's tenancy — this is the right answer	Nothing; this is the one to aim for
Full OIDC in the app	1-2 days	Production	A three-day prototype

Shared password gate in Streamlit

PYTHON

```python
# ui/auth.py
import hmac
import streamlit as st


def check_password() -> bool:
    """Minimal gate. Blocks casual access; it is NOT real security.
    Use it with HTTPS and a non-guessable password, and never with real PII."""

    def entered():
        if hmac.compare_digest(st.session_state["pw"], st.secrets["APP_PASSWORD"]):
            st.session_state["authenticated"] = True
            del st.session_state["pw"]                # never keep it in state
        else:
            st.session_state["authenticated"] = False

    if st.session_state.get("authenticated"):
        return True

    st.title("Policy Assistant")
    st.caption("Internal prototype. Access is restricted.")
    st.text_input("Password", type="password", on_change=entered, key="pw")
    if st.session_state.get("authenticated") is False:
        st.error("Incorrect password")
    return False


# ui/app.py
# if not check_password():
#     st.stop()
```

HTTP Basic and API keys in FastAPI

PYTHON

```python
import secrets
from typing import Annotated

from fastapi import Depends, HTTPException, status
from fastapi.security import APIKeyHeader, HTTPBasic, HTTPBasicCredentials

basic = HTTPBasic()

def require_basic(creds: Annotated[HTTPBasicCredentials, Depends(basic)]) -> str:
    settings = get_settings()
    user_ok = secrets.compare_digest(creds.username, settings.basic_user)
    pass_ok = secrets.compare_digest(creds.password,
                                     settings.basic_password.get_secret_value())
    if not (user_ok and pass_ok):
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid credentials",
                            headers={"WWW-Authenticate": "Basic"})
    return creds.username


api_key_header = APIKeyHeader(name="X-API-Key", auto_error=False)

def require_api_key(key: Annotated[str | None, Depends(api_key_header)]) -> str:
    """Per-user keys so you can attribute and revoke individually."""
    keys = get_settings().api_keys          # {"key-abc": "sunita@customer.com", ...}
    if not key or key not in keys:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Invalid or missing API key"
    return keys[key]


# Signed, expiring link - good for "here is your report, valid 24 hours"
import hmac, hashlib, time, base64

def sign_link(resource_id: str, ttl_s: int = 86400) -> str:
    secret = get_settings().link_secret.get_secret_value().encode()
    expires = int(time.time()) + ttl_s
    payload = f"{resource_id}:{expires}".encode()
    sig = base64.urlsafe_b64encode(hmac.new(secret, payload, hashlib.sha256).digest())
    return f"/shared/{resource_id}?expires={expires}&sig={sig}"

def verify_link(resource_id: str, expires: int, sig: str) -> bool:
    if time.time() > expires:
        return False
    secret = get_settings().link_secret.get_secret_value().encode()
    payload = f"{resource_id}:{expires}".encode()
    expected = base64.urlsafe_b64encode(hmac.new(secret, payload, hashlib.sha256).dige
    return hmac.compare_digest(sig, expected)
```

OAuth proxy — the one to aim for

Put the app behind a proxy that handles the customer's identity provider. Your app stays auth-free and receives a trusted header with the user's identity. This is a configuration task, not a development task, and it is what the customer's security team actually wants to see.

YAML

```yaml
# docker-compose.yml fragment: oauth2-proxy in front of the Streamlit app
services:
  oauth2-proxy:
    image: quay.io/oauth2-proxy/oauth2-proxy:latest
    command:
      - --provider=oidc
      - --oidc-issuer-url=https://login.microsoftonline.com/${TENANT_ID}/v2.0
      - --client-id=${CLIENT_ID}
      - --client-secret=${CLIENT_SECRET}
      - --cookie-secret=${COOKIE_SECRET}
      - --email-domain=customer.com
      - --upstream=http://ui:8501
      - --http-address=0.0.0.0:4180
      - --pass-authorization-header=true
      - --set-xauthrequest=true
      - --skip-provider-button=true
    ports: ["443:4180"]
    depends_on: [ui]

  ui:
    build: .
    command: streamlit run ui/app.py --server.port 8501 --server.address 0.0.0.0
    expose: ["8501"]
    # No ports: - the ONLY way in is through the proxy.
```

PYTHON

```
# On Azure Container Apps / App Service, "Easy Auth" does the same with no container:
#   az containerapp auth microsoft update -g rg -n myapp \
#     --client-id $CLIENT_ID --client-secret $CLIENT_SECRET \
#     --tenant-id $TENANT_ID --yes
#   az containerapp auth update -g rg -n myapp --action Return401 --enabled true
#
# Your app then reads the identity from injected headers:
user_email = request.headers.get("X-MS-CLIENT-PRINCIPAL-NAME")     # Azure Easy Auth
user_email = request.headers.get("X-Auth-Request-Email")           # oauth2-proxy
user_email = request.headers.get("X-Goog-Authenticated-User-Email") # Google IAP
```

The rule you must never break

Customer data must never be reachable from the public internet without authentication. Not "probably nobody will find the URL". Not "it's only for a week". Internet-wide scanners find new hosts within minutes, and a Streamlit app with a file browser or a vector index full of contracts is a breach you will have to report. If you need remote access and cannot stand up a proxy in time, the correct answer is a screen share — not a public URL. This is one of the few genuinely non-negotiable rules in prototyping.

Packaging and Deployment

"It works on my laptop" is fine for the first demo and nowhere near enough for the second. The moment a customer wants to click around on their own, you need it running somewhere they can reach, in their tenancy, with their secrets. Two Dockerfiles and a compose file get you 90% of the way, and the same artefacts carry you into production later.

Dockerfile for the FastAPI backend

DOCKERFILE

```dockerfile
# Multi-stage: build deps once, ship a small runtime image.
FROM python:3.12-slim AS builder

ENV UV_COMPILE_BYTECODE=1 UV_LINK_MODE=copy

COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev --no-install-project

COPY app ./app
RUN uv sync --frozen --no-dev


FROM python:3.12-slim AS runtime

RUN apt-get update \

 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/* \
 && useradd --create-home --uid 10001 appuser

WORKDIR /app
COPY --from=builder --chown=appuser:appuser /app/.venv /app/.venv
COPY --chown=appuser:appuser app ./app

ENV PATH="/app/.venv/bin:$PATH" \
    PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1

USER appuser
EXPOSE 8000

HEALTHCHECK --interval=30s --timeout=5s --start-period=40s --retries=3 \

  CMD curl -fsS http://localhost:8000/health || exit 1

# One worker per container; scale by running more containers.
# --proxy-headers is required behind a load balancer or your URLs will be wrong.
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", \

     "--workers", "1", "--proxy-headers", "--forwarded-allow-ips", "*"]
```

Dockerfile for the Streamlit frontend

DOCKERFILE

```
FROM python:3.12-slim

COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv

RUN apt-get update \

 && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/* \
 && useradd --create-home --uid 10001 appuser

WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev
COPY --chown=appuser:appuser ui ./ui

ENV PATH="/app/.venv/bin:$PATH" \
    PYTHONUNBUFFERED=1 \
    STREAMLIT_SERVER_HEADLESS=true \
    STREAMLIT_BROWSER_GATHER_USAGE_STATS=false \
    STREAMLIT_SERVER_FILE_WATCHER_TYPE=none

USER appuser
EXPOSE 8501

HEALTHCHECK --interval=30s --timeout=5s --start-period=20s --retries=3 \

  CMD curl -fsS http://localhost:8501/_stcore/health || exit 1

CMD ["streamlit", "run", "ui/app.py", \

     "--server.port=8501", "--server.address=0.0.0.0", \
     "--server.enableCORS=false", "--server.enableXsrfProtection=true"]
```

docker-compose for the whole thing

YAML

```yaml
services:
  api:
    build: {context: ., dockerfile: Dockerfile.api}
    environment:
      OPENAI_API_KEY: ${OPENAI_API_KEY:?set OPENAI_API_KEY in .env}
      VECTOR_STORE_PATH: /data/index
      MODEL_NAME: ${MODEL_NAME:-gpt-4o-mini}
      DEMO_MODE: ${DEMO_MODE:-false}
      LOG_LEVEL: ${LOG_LEVEL:-INFO}
    volumes:
      - appdata:/data
    expose: ["8000"]
    healthcheck:
      test: ["CMD", "curl", "-fsS", "http://localhost:8000/health"]
      interval: 30s
      timeout: 5s
      retries: 3
    deploy:
      resources:
        limits: {cpus: "2.0", memory: 4G}
    restart: unless-stopped

  ui:
    build: {context: ., dockerfile: Dockerfile.ui}
    environment:
      API_URL: http://api:8000              # service name, not localhost
      API_KEY: ${API_KEY:?set API_KEY in .env}
    ports: ["8501:8501"]
    depends_on:
      api: {condition: service_healthy}
    restart: unless-stopped

volumes:
  appdata:
```

⚠ **Two compose mistakes that cost an hour each**

1. The UI calling `http://localhost:8000` — inside a container, `localhost` is that container. Use the service name (`http://api:8000`). **2.** Forgetting a volume for the index, so every restart loses the data you spent twenty minutes ingesting, including five minutes before a demo. Name your volumes and test `docker compose down && docker compose up` before you rely on it.

Where to deploy

Target	Time to deploy	Good for	Watch out for
Azure Container Apps	~15 min	Azure customers; built-in HTTPS, Easy Auth, scale-to-zero, secrets	Scale-to-zero means cold starts; set min replicas to 1 before a demo
Google Cloud Run	~10 min	GCP customers; simplest container-to-URL path there is	Request timeout cap; needs explicit config for long streams
AWS App Runner / ECS Fargate	20-40 min	AWS customers	App Runner is simple but limited; ECS needs more wiring
Hugging Face Spaces	~5 min	Public model demos, Gradio especially	**Never** for customer data — it is third-party hosting
Customer VM + docker compose	30-60 min	Air-gapped, strict data residency, "it must stay inside"	You now own patching, TLS, restarts; use `restart: unless-stopped` and a systemd unit
Streamlit Community Cloud	~5 min	Public demos of non-sensitive apps	Public by default; resource limits; not for customer data

BASH

```
# Azure Container Apps - the fastest credible path for an enterprise customer
az group create -n rg-demo -l westeurope

az containerapp env create -n demo-env -g rg-demo -l westeurope

# Secrets go in the platform, never in the image
az containerapp create \
  -n doc-assistant-api -g rg-demo --environment demo-env \
  --image myacr.azurecr.io/doc-api:0.1.0 \
  --target-port 8000 --ingress internal \
  --min-replicas 1 --max-replicas 3 \
  --cpu 1.0 --memory 2Gi \
  --secrets openai-key=$OPENAI_API_KEY \
  --env-vars OPENAI_API_KEY=secretref:openai-key VECTOR_STORE_PATH=/data/index

az containerapp create \
  -n doc-assistant-ui -g rg-demo --environment demo-env \
  --image myacr.azurecr.io/doc-ui:0.1.0 \
  --target-port 8501 --ingress external \
  --min-replicas 1 --max-replicas 1 \
  --env-vars API_URL=https://doc-assistant-api.internal.<env-domain>

# Lock it down with the tenant's identity provider
az containerapp auth microsoft update -g rg-demo -n doc-assistant-ui \
  --client-id $CLIENT_ID --client-secret $CLIENT_SECRET --tenant-id $TENANT_ID --yes
az containerapp auth update -g rg-demo -n doc-assistant-ui \
  --action RedirectToLoginPage --enabled true
```

Streamlit does not scale horizontally without sticky sessions

Session state lives in one process's memory. Two replicas behind a round-robin load balancer means a user's second click may hit a different replica with no memory of them, and the app appears to reset randomly. For a prototype, pin `min-replicas = max-replicas = 1` and move on. If you genuinely need more capacity, that is one of the signals that it is time to graduate to a real frontend.

Resource sizing for a prototype

Component	Starting point	Increase when
FastAPI backend (API-based LLM)	1 vCPU, 1-2 GB	You see memory pressure from large document parsing
FastAPI with a local embedding model	2 vCPU, 4 GB	Embedding batches get slow; consider GPU only if you must
Streamlit UI	0.5-1 vCPU, 1 GB	Rarely — the UI is thin if you kept logic in the backend
Vector index (Chroma/FAISS on disk)	Index size × ~1.5 in RAM	Beyond ~1M chunks, move to a managed vector service
Disk	10-20 GB	Uploads accumulate; add a cleanup job

Observability and Feedback in a Prototype

A prototype with no logging teaches you nothing. A prototype that captures every question, every retrieval, every latency and every thumbs-down turns a one-hour demo into a labelled dataset you can use to improve the system and to justify the next phase. This is the cheapest high-leverage thing in the entire guide, and almost nobody does it.

Structured request logging

PYTHON

```python
# app/middleware.py
import json, logging, time, uuid

from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware

log = logging.getLogger("requests")


class RequestLogMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):
        rid = request.headers.get("x-request-id") or str(uuid.uuid4())[:8]
        request.state.request_id = rid
        t0 = time.perf_counter()

        try:
            response = await call_next(request)
            status = response.status_code
        except Exception:
            status = 500
            raise
        finally:
            log.info(json.dumps({
                "request_id": rid,
                "method": request.method,
                "path": request.url.path,
                "status": status,
                "duration_ms": round((time.perf_counter() - t0) * 1000, 1),
                "user": request.headers.get("x-auth-request-email", "anonymous"),
                "ua": request.headers.get("user-agent", "")[:80],
            }))

        response.headers["X-Request-ID"] = rid
        return response
```

PYTHON

```python
# Log the AI-specific things a request log does not capture.
# These five fields are what let you debug "why was that answer wrong?" afterwards.
def log_interaction(request_id, question, retrieved, answer, model, latency_ms, cost_u
    log.info(json.dumps({
        "event": "ask",
        "request_id": request_id,
        "question": question,
        "retrieved_ids": [c.chunk_id for c in retrieved],
        "retrieved_scores": [round(c.score, 3) for c in retrieved],
        "top_score": round(retrieved[0].score, 3) if retrieved else None,
        "answer_chars": len(answer),
        "said_not_found": "could not find that" in answer.lower(),
        "model": model,
        "latency_ms": latency_ms,
        "cost_usd": cost_usd,
    }))
```

Capturing the eval set from the demo itself

✓ **The move that pays for itself twice**

Every question a customer asks during a demo is a real user query, which is exactly what you cannot invent. Log all of them. After a one-hour session you have 30-60 genuine questions, the retrieved passages for each, and often a thumbs-up/down. That is your first evaluation set, and it is far more valuable than anything you would have written yourself. Export it at the end of the day, send it to them in the follow-up email as "the questions you asked and how the system handled them", and you have simultaneously built a test suite and demonstrated rigour.

PYTHON

```python
# scripts/export_eval.py
import json, sqlite3
from pathlib import Path

def export(db="data/app.db", out="data/eval_set.jsonl"):
    c = sqlite3.connect(db)
    rows = c.execute("""
        select q.request_id, q.ts, q.question, q.answer, q.chunk_ids, q.latency_ms,
               f.rating, f.comment
        from queries q
        left join feedback f on f.request_id = q.request_id
        order by q.ts
    """).fetchall()

    with Path(out).open("w", encoding="utf-8") as fh:
        for rid, ts, question, answer, chunk_ids, latency, rating, comment in rows:
            fh.write(json.dumps({
                "id": rid, "ts": ts,
                "question": question,
                "generated_answer": answer,
                "retrieved": json.loads(chunk_ids),
                "latency_ms": latency,
                "human_rating": rating,
                "human_comment": comment,
                "expected_answer": None,     # fill these in with the customer's SME
            }) + "\n")

    neg = sum(1 for r in rows if r[6] == -1)
    print(f"Exported {len(rows)} interactions ({neg} negative) to {out}")

if __name__ == "__main__":
    export()
```

What to measure, even in a prototype

Metric	Why it matters in a demo context
p50 / p95 latency, end to end	The customer will ask "how fast is it?" — have a number, not an impression
Time to first token	The number that determines perceived speed; optimise this over total latency
Retrieval top score distribution	Consistently low top scores means your chunking or embedding is wrong, not the model
"Could not find" rate	High rate = coverage gap. This is a concrete, fixable finding to report.
Thumbs-down rate and the questions behind it	Your prioritised backlog, written by the customer
Cost per question	Lets you answer "what would this cost for 500 users?" credibly
Error rate by type	Tells you which failure mode to harden first

PYTHON

```python
# A tiny cost tracker. Verify current prices before quoting anything.
PRICES_PER_1K = {                    # USD per 1k tokens — CHECK CURRENT PRICING
    "gpt-4o-mini":           {"in": 0.00015, "out": 0.0006},
    "gpt-4o":                {"in": 0.0025,  "out": 0.01},
    "text-embedding-3-small": {"in": 0.00002, "out": 0.0},
}

def estimate_cost(model: str, in_tokens: int, out_tokens: int) -> float:
    p = PRICES_PER_1K.get(model)
    if not p:
        return 0.0
    return (in_tokens / 1000) * p["in"] + (out_tokens / 1000) * p["out"]

# Then in the follow-up email:
#   "Average cost per question in the session was $0.0031.
#    At 200 questions/day that is about $19/month in model spend."
# A concrete number like that moves a budget conversation forward more
# than any amount of architecture discussion.
```

Performance Tricks

A prototype does not need to be fast in the throughput sense — it needs to *feel* fast to one person clicking through it. Those are different problems with different solutions, and the handful of techniques below cover almost all of the perceived-latency wins available to you.

Rank the wins

Technique	Typical win	Effort
Stream the response instead of waiting	8 s → 400 ms *perceived*	30 min
Load models/clients once at startup, not per request	Removes 2-8 s from every request	10 min
Cache answers for repeated/demo questions	6 s → 20 ms	20 min
Warm up before the demo	Removes the first-request penalty entirely	5 min
Batch embedding calls	5-20x on ingestion	15 min
Use a smaller/faster model for the demo path	2-4x, often with no visible quality loss	5 min
Show progress for anything over ~1 s	No real win; large perceived win	5 min
Reduce retrieved chunks from 10 to 5	~30% fewer input tokens, faster and cheaper	1 min

Warm start and lazy loading

PYTHON

```python
from contextlib import asynccontextmanager

@asynccontextmanager
async def lifespan(app):
    s = get_settings()

    # 1. Load heavy objects ONCE
    app.state.index = VectorIndex.load(s.vector_store_path)
    app.state.llm   = LLMClient(s.openai_api_key.get_secret_value(), s.model_name)

    # 2. Warm the embedding model with a throwaway call so the FIRST real
    #    user request does not pay for lazy initialisation inside the library.
    try:
        app.state.index.search("warmup", k=1)
        log.info("index warm")
    except Exception as e:
        log.warning("warmup failed (continuing): %s", e)

    # 3. Optionally pre-warm the model connection pool
    try:
        await asyncio.wait_for(app.state.llm.ping(), timeout=10)
    except Exception:
        log.warning("model not reachable at startup - demo mode may be needed")

    yield
    await app.state.llm.aclose()
```

PYTHON

```python
# Lazy-load things that are expensive AND not always needed.
# This keeps container startup fast (important for scale-to-zero platforms)
# while still paying the cost only once.
from functools import lru_cache

@lru_cache(maxsize=1)
def get_reranker():
    from sentence_transformers import CrossEncoder
    return CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")

# Called only when rerank=True; the import itself costs ~2 s.
```

Batching

PYTHON

```python
async def embed_many(texts: list[str], batch_size: int = 256) -> list[list[float]]:
    """One API call per 256 texts instead of one per text.
    On a 10k-chunk corpus this is the difference between 20 minutes and 90 seconds."""
    out: list[list[float]] = []
    for i in range(0, len(texts), batch_size):
        batch = texts[i:i + batch_size]
        r = await client.post("/embeddings",
                              json={"model": "text-embedding-3-small", "input": batch}
        r.raise_for_status()
        out.extend(d["embedding"] for d in sorted(r.json()["data"], key=lambda d: d["i
    return out
```

Cold start on serverless platforms

> ⚠ **Scale-to-zero and demos do not mix**
>
> Cloud Run and Container Apps can scale to zero, which is excellent for cost and terrible for a demo: the first request after idle waits for a container start plus your model load — commonly 15-40 seconds, during which the customer concludes the system is broken. Before any demo, set minimum replicas to 1, and add a keep-warm ping every few minutes. Afterwards, set it back to zero. Put both commands in your runbook so you cannot forget.

BASH

```bash
# Before the demo
az containerapp update -n doc-assistant-api -g rg-demo --min-replicas 1
az containerapp update -n doc-assistant-ui  -g rg-demo --min-replicas 1

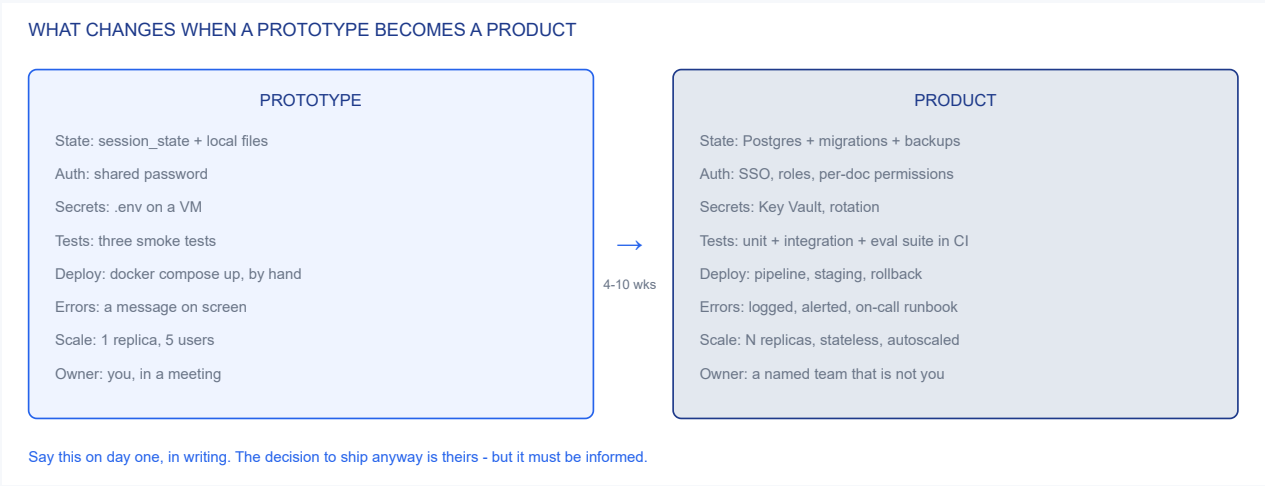
# Keep-warm from anywhere (cron, GitHub Actions, your laptop)
while true; do curl -fsS https://doc-assistant-api ... /health > /dev/null; sleep 240; d

# After the demo
az containerapp update -n doc-assistant-api -g rg-demo --min-replicas 0
```

The Graduation Path

At some point the prototype gets used by real people, and everything you deliberately skipped becomes a liability. This section is the honest list of what changes, so you can

estimate it, price it, and have the conversation before the customer discovers it the hard way.



The graduation gap. Estimate it once and reuse the estimate; it is remarkably consistent across projects.

The nine things that must change

1. **State moves out of the process.**
session_state and local files become Postgres (or the customer's standard store) with schema migrations via Alembic, backups, and a restore you have actually tested. Until this is done the app cannot be restarted safely or scaled at all.

2. **Real authentication and authorisation.**
SSO through their IdP, roles, and — critically for document systems — per-document permissions enforced at retrieval time, not just at the UI. A RAG system that ignores source ACLs is a data-leak machine.

3. **Secrets management.**
`.env` becomes Key Vault / Secret Manager / Parameter Store, injected at runtime, with rotation and an audit trail. No secret in an image, a repo or an environment variable printed in logs.

4. **Tests.**
Unit tests on the services, integration tests on the API, and an evaluation suite that runs your golden question set and asserts quality thresholds — because a prompt change can silently degrade answers in a way no unit test catches.

5. **CI/CD.**

Build, test, scan, push, deploy to staging, smoke test, promote. Plus a rollback that someone has performed at least once as a drill.

6 **Observability.**
Structured logs shipped somewhere searchable, metrics with dashboards, traces for the slow path, alerts with an on-call rotation, and an SLO everyone has agreed to.

7 **Scaling and resilience.**
Stateless replicas behind a load balancer, autoscaling rules, graceful shutdown, circuit breakers on the model provider, and a defined behaviour when the provider is down.

8 **A frontend that is not Streamlit.**
If more than a handful of people use it daily, you will hit the limits of the rerun model, the lack of sticky sessions, and the styling ceiling. Budget for React or their standard frontend stack.

9 **Ownership.**
A named team, documentation they wrote or reviewed, a runbook they have exercised, and an agreed support model. Without this the system decays regardless of its technical quality.

The prototype debt register

Start this file on day one and update it every time you take a shortcut. It takes thirty seconds per entry and it is the single most useful artefact you hand over.

TEXT

PROTOTYPE DEBT REGISTER — Document Assistant — updated 2026-03-14

ID	AREA	SHORTCUT TAKEN	RISK IF SHIPPED AS-IS	E
D1	Auth	Shared password in st.secrets	Anyone with the URL+pw sees all indexed documents	
D2	AuthZ	No per-document permissions	Users see documents they are not entitled to. BLOCKER.	
D3	State	session_state + SQLite on a volume	Restart loses conversations; cannot scale past 1 replica	
D4	Secrets	OPENAI_API_KEY in container env	No rotation, visible to anyone with container access	
D5	Ingestion	Manual script, run by hand	New documents are not indexed until someone remembers	
D6	Ingestion	Scanned PDFs skipped entirely	~30% of the corpus is invisible to the assistant. Needs OCR.	
D7	Quality	No evaluation suite	Prompt changes can silently degrade answers	
D8	Ops	No logging/alerting beyond stdout	Failures are invisible	
D9	Deploy	Deployed by hand with az cli	No rollback, no staging	
D10	Frontend	Streamlit, 1 replica, no sticky sessions	Breaks above ~10 concurrent users	

BLOCKERS (D2):	6d
MINIMUM VIABLE PRODUCTION:	31d
FULL:	51d

Assumption: one engineer. Excludes customer-side security review and procurement, which historically add 2–6 weeks of elapsed time.

✓ **Why this document wins arguments**

When the customer asks "can't we just use the prototype?", you do not have to say no, and you do not have to be vague. You show the register, point at D2 as a blocker, and offer a six-day path to something defensible. You have turned an uncomfortable conversation into a scoping exercise with a number attached — which is exactly what the person holding the budget needs.

Anti-Patterns and How Prototypes Fail

These are the specific, repeated ways prototypes fail in front of customers. Each one is cheap to avoid and expensive to experience.

THE INFINITE PROTOTYPE

Four months in, still "a prototype", now with twelve features and six users, no tests, and nobody willing to own it. Caused by never defining what "done" means. Fix: a written scope and end date before you start, and a go/no-go decision meeting at the end.

THE DEMO-ONLY PATH

It works beautifully for the three questions you rehearsed and crashes on the fourth. Caused by testing only the script. Fix: have someone who has not seen it use it for ten minutes before the demo, and fix whatever they break.

THE SYNTHETIC-DATA ILLUSION

Built on clean generated data, it falls apart on the customer's actual records with their missing fields, duplicate ids and 1990s encodings. Fix: insist on real (masked) data in week one, even if it is only 500 rows.

THE HIDDEN STUB

You stubbed the write-back and forgot to mention it. The customer thinks it is working end to end and tells their boss. Later this looks like dishonesty. Fix: say every stub out loud during the demo and list them in the follow-up email.

OVER-ENGINEERING ON DAY ONE

Three days spent on Kubernetes manifests, a message queue and a CI pipeline for a demo seen by four people once. Fix: the infrastructure rule — nothing that is not needed for this week's demo.

THE SILENT FAILURE

The model call fails, the code swallows the exception, and the UI shows an empty answer. The customer concludes the model is stupid rather than that the network is broken. Fix: never catch broadly without surfacing something to the user.

CHERRY-PICKED EXAMPLES

You only ever show the five documents you know retrieve well. Someone picks a sixth and it fails. Fix: test against a random sample before the demo and know your actual hit rate — then you can say "it handles roughly 8 in 10 of these; here's what the failures look like", which is far more credible.

THE UNREPRODUCIBLE ENVIRONMENT

It runs on your laptop and nowhere else, because of an unpinned dependency, a local file path, or a tool you installed six months ago. Fix: build the Docker image on day one and do your own testing through it.

✕ **The failure that ends engagements**

A prototype containing the customer's confidential documents left running on a public URL after the demo. Nobody turned it off; a scanner found it; the customer's security team found out from someone else. Everything you built becomes irrelevant. Have a written teardown step: when the demo period ends, the app is stopped, the data is deleted, the keys are rotated, and you send a short email confirming all three. Put it in your calendar before the demo, not after.

Interview Questions

A customer wants a demo in three days of an AI assistant over their internal documents. Walk me through your plan.

Day zero, before anything technical: I ask for a sample of the documents and confirm in writing whether that data may be sent to a hosted model API, because the answer determines the entire architecture and I cannot un-ask it later. Day one: ingestion, because that is where the surprises are. Extract text from the real files, discover which are scanned images, chunk, embed, index, and run ten questions through retrieval alone to see whether the right passages come back — if retrieval is bad the model cannot save it. Day two: FastAPI backend with a streaming endpoint and a Streamlit frontend with chat, sources panel, upload and a reset button, plus structured logging of every query. Day three: harden the demo — seed data, populate a demo cache by running the full script once, test on the actual network, write the pre-demo checklist, and spend an hour letting a colleague try to break it. I would deliberately not build authentication beyond a password gate, not deploy to the cloud unless they need remote access, and not add any feature beyond the single workflow we agreed. Everything I skipped goes in a debt register that I hand over with the demo.

Why would you build a FastAPI backend rather than putting everything in Streamlit?

Three reasons, and the first is the decisive one. First, reusability: when the customer says "actually we want this inside our existing portal" or "can it run as a batch job over the whole queue", a FastAPI backend already answers both and only the thin UI is thrown away. If the logic lives inside Streamlit callbacks, that request means a rewrite. Second, testability: service functions that take plain arguments and return plain objects can be tested in two seconds with pytest, with no browser and no server, which matters enormously when you are changing things an hour before a demo. Third, concurrency and streaming: Streamlit's one-script-per-session model makes real concurrency control, background jobs and token streaming awkward, whereas FastAPI gives you async, semaphores, SSE and proper timeouts. The cost is about two hours on day one plus one extra container. In the rare case where the thing genuinely is a single-user notebook replacement that will never be integrated, I would skip it and say so.

Explain Streamlit's execution model and the most common bug it causes.

Streamlit re-runs the entire script from top to bottom on every interaction. There is no component tree and no persistent handler scope — when you click a button, your script executes again from line one, and each widget function returns its current value. The most common bug is losing state: people store conversation history or a loaded model in a local variable and are confused when it resets on every click, because that variable is recreated each run. The fix is `st.session_state` for per-session data, initialised behind an `if key not in session_state` guard so it is set once, and the caching decorators for expensive objects — `cache_resource` for models, clients and connections which are shared by reference, and `cache_data` for serialisable results which are copied per caller. The second-most-common bug follows from the same model: loading a model at module level without caching, so every single keystroke in a text input reloads it and the app feels broken. Once you internalise "the script reruns, so anything that must survive goes in session_state or a cache", the framework stops being surprising.

What is the difference between `async def` and `def` in a FastAPI route, and when does it bite you?

An `async def` handler runs directly on the event loop, which is correct and efficient when everything slow inside it is awaited. A plain `def` handler is run in a threadpool, which is safe for blocking I/O at the cost of being bounded by the thread count. The bite is putting blocking code inside `async def` — calling `requests.get`, `time.sleep`, a synchronous database driver, or doing heavy CPU work. That blocks the event loop, so every other request handled by that worker stalls until it finishes. In single-user testing you never notice; in a demo where the customer opens three tabs, everything appears to hang and it looks like the model is broken. The rules I apply: use `async def` with async clients like httpx for I/O; use plain `def` if you want to keep a synchronous library; push CPU-bound work to `run_in_threadpool` or a process pool; and if unsure, write `def`, because the threadpool is marginally slower in theory and dramatically safer in practice.

How do you make a prototype resilient to a bad network during a customer demo?

Layered defence. First, a demo cache: I run the full demo script once on a good connection the day before, which stores the answer for every planned question keyed on a normalised hash, so on the day those questions return in milliseconds with no network at all. Second, graceful degradation: a short timeout on the model call — around ten seconds rather than sixty — falling back to the cache, and failing that to a message plus the retrieved passages, so the customer still sees the retrieval working even if generation fails. Third, no stack traces ever reach the screen; there is a catch-all handler that renders a readable message with the technical detail hidden behind an expander. Fourth, operational preparation: a tethered mobile hotspot tested in advance, minimum replicas set to one so there is no cold start, a warm-up request fired before they walk in, and a screen recording of the happy path saved locally. And finally a rehearsed sentence for when something fails anyway, because the recovery is what the room remembers, not the failure.

A customer wants to put your prototype straight into production. How do you respond?

I do not refuse, because refusing makes me an obstacle and they may have a legitimate reason to accept the risk. Instead I make the risk explicit and priced. I show the prototype debt register — which I started on day one — and separate it into three tiers: blockers that create unacceptable risk, items needed for minimum viable production, and nice-to-haves. For a document assistant the blocker is almost always authorisation: the prototype does not enforce per-document permissions, so users can retrieve content they are not entitled to, which is a data-leak risk regardless of how good the answers are. I would quantify that as, say, six days of work, and the minimum viable production path as around four to six weeks including auth, durable state, secrets management, an evaluation suite, deployment automation and monitoring. Then I let them choose. Very often they pick a middle path: fix the blockers, run a limited pilot with twenty named users, and schedule the rest. What matters is that the decision is theirs and informed, and that it is in writing — because in six months somebody will ask why there are no tests.

How do you handle authentication for a prototype that needs to be accessible to the customer?

The governing rule is that customer data is never reachable from the public internet without authentication — not "probably nobody will find it", because scanners find new hosts within minutes. Within that constraint I scale the effort to the context. If it only needs to be seen in a meeting, I screen-share from localhost and there is no auth problem to solve. If a handful of named people need to click around, a shared password gate plus HTTPS is defensible for non-sensitive data. The answer I actually push for, because it takes one to two hours and is what their security team wants to see, is an OAuth proxy in front of the app — oauth2-proxy in compose, or Azure Easy Auth, or Google IAP — which handles their identity provider entirely outside my code and passes the authenticated email in a header. My app stays auth-free and I get real SSO and an audit trail. What I would never do is a public Gradio share link or a cloud endpoint with open ingress containing their documents, because that is a reportable incident rather than a shortcut.

What would you log in a prototype, and why bother?

I log every interaction with the question, the retrieved chunk ids and their scores, the generated answer, the model, latency, and any thumbs-up or thumbs-down — into SQLite, which is one file I can

hand over. The reason is that a demo is the only time you get real user queries before launch, and you cannot invent them. After a one-hour session you have thirty to sixty genuine questions with retrieval traces and partial human judgements, which is your first evaluation set and is far better than anything you would have written. It also lets you answer, concretely, the questions the customer will ask next: what was the p95 latency, what fraction of questions could not be answered from the corpus, which documents were never retrieved, what did a question cost on average. Sending that analysis in the follow-up email — "here are the forty questions you asked, here's where it struggled, here's what that implies for scope" — does more for the next phase than any slide. It costs about twenty lines of code.

Your demo app is slow — eight seconds per answer. What do you do?

First I measure where the time goes rather than guessing: retrieval, model generation, and any pre- or post-processing, timed separately. Usually generation dominates, in which case the highest-impact fix is not making it faster but making it *feel* faster by streaming — time to first token of a few hundred milliseconds with text flowing reads as instant, even though total latency is unchanged. That is a thirty-minute change and it is the single biggest perceived win available. After that: check that models, clients and the index are loaded once in the lifespan rather than per request, which often removes several seconds that nobody noticed were there; cache answers for repeated questions; reduce the number of retrieved chunks, since going from ten to five cuts input tokens by roughly half with little quality loss in most corpora; batch anything that loops over items; and consider a smaller model for the demo path. If retrieval is the slow part, the index is usually either too large for the machine or doing something pathological like re-embedding the query with a cold model. I would also set minimum replicas to one so there is no cold start, which is otherwise the worst eight seconds of the demo.

When would you choose Gradio over Streamlit?

Gradio when the app is fundamentally a model with inputs and outputs, and especially when the inputs or outputs are images, audio or video, because its components for those are much better. Also when I want a chat UI in under ten lines with streaming, cancellation and a queue already handled — `ChatInterface` does in five lines what takes thirty in Streamlit. Also when deploying to Hugging Face Spaces, which is Gradio's native home, or when I want the UI to double as a callable API, which Gradio exposes automatically. Streamlit when the app is more of an internal business tool — multiple pages, dataframes, filters, metrics, a sidebar of controls — because it has better primitives for data display and navigation and it looks more like software the customer's staff already use. For typical enterprise FDE work the answer is Streamlit more often than not, because the demos are usually chat plus a table plus an upload. The one thing I would never do is choose Gradio because of the `share=True` link when the app contains customer data, since that is a public tunnel with no authentication.

How do you keep a three-day prototype from becoming unmaintainable?

One structural rule does most of the work: no business logic inside a framework callback. Everything real — retrieval, prompt construction, model calls, parsing, validation — lives in plain functions in a services module that imports neither FastAPI nor Streamlit. The route handlers and the UI only call

> those functions and display results. That single discipline means the logic is testable without a server, reusable from a script or a notebook, and survives replacing the UI. On top of that: pin dependencies with a lockfile so the environment is reproducible, keep configuration in one pydantic-settings class rather than scattered `os.getenv` calls, write three tests covering the happy path, a validation failure and an upstream failure, and build the Docker image on day one so you discover environment problems immediately rather than the night before. None of this is heavyweight — it is maybe two extra hours across three days — and it is the difference between a prototype that can be graduated and one that can only be rewritten.

> **What would make you refuse to build a prototype?**
>
> A few situations. If the data required does not exist and will not exist within the engagement, a prototype would have to run on fiction, and a demo on synthetic data proves nothing to a room full of people who know their data is messy. If legal or privacy has not cleared sending the data to the model provider I planned to use, and there is no viable on-premise or in-tenancy alternative within scope, I would stop and resolve that first rather than build something I may have to delete. If the real question is organisational — will people actually change their workflow — a prototype cannot answer it, and I would propose a different intervention such as shadowing the current process. And if they are describing a production system but calling it a prototype to bypass procurement and security review, I would clarify that explicitly, because building to prototype standards something that will be used in production is how people get hurt. In every one of these cases the right move is to say what I think is actually needed, propose the smaller thing that would genuinely de-risk the decision, and get agreement in writing.

Build-a-Demo Lab Plan and Checklists

The only way to get fast at this is to do it repeatedly under a clock. Each lab below is deliberately timeboxed; the discipline of stopping when the box ends is as much of the skill as the code.

Lab 1 — Four hours: the baseline demo

1. **00:00-00:20**
   — `uv init`, pin Python, add FastAPI, Streamlit, httpx, pydantic-settings. Create the folder layout. Commit.
2. **00:20-01:00**
   — Backend: `/health`, `/ask` returning a hardcoded answer, settings class, three pytest tests with fakes. Prove the tests run in under five seconds.
3. **01:00-01:40**

— Frontend: chat UI calling the backend. Sidebar with health status. Reset button. The *entire* frontend talks to the backend through one `api_client.py`.

4. **01:40-02:30**
— Make it real: ingest 20 documents you actually have, wire retrieval, wire a model. Fix whatever breaks.

5. **02:30-03:00**
— Add streaming end to end. Measure time to first token before and after; write both numbers down.

6. **03:00-03:30**
— Harden: catch-all error handler, friendly error states, loading indicators, demo cache, seed script.

7. **03:30-04:00**
— Dockerfiles and compose. Run `docker compose down && up` and confirm the index survives. Write a five-line README.

Lab 2 — Two hours: deploy and lock down

1. Push both images to a registry.
2. Deploy to Container Apps or Cloud Run with internal ingress on the API and external on the UI.
3. Put secrets in the platform's secret store; confirm nothing sensitive is in the image (`docker history` and a `grep` of the layers).
4. Add an OAuth proxy or the platform's built-in auth. Confirm that an unauthenticated request gets a 401 — actually test it from a private browser window.
5. Verify streaming still works through the load balancer. If it buffers, fix the headers. This is the step everyone skips and everyone regrets.
6. Set minimum replicas to one, measure cold vs warm first-request latency, write both into your runbook.
7. Write the teardown script: stop, delete data, rotate keys.

Lab 3 — One hour: break it on purpose

1. Disconnect the network mid-answer. What does the user see? Fix it.
2. Set the model API key to garbage. What does the user see? Fix it.
3. Upload a 40 MB file, a zero-byte file, a `.exe` renamed to `.pdf`, and a scanned PDF.
4. Paste 50,000 characters into the question box.
5. Open five tabs and submit simultaneously. Does anything block? If yes, you have a blocking call inside `async def`.
6. Restart the container mid-conversation. What is lost? Is that acceptable? Write it in the debt register.
7. Ask ten questions you know are not covered by the corpus. Does it hallucinate or say it cannot find them? This is the behaviour the customer will test first.

Lab 4 — Repeat under pressure

1. Pick a dataset or document corpus you have never seen. Give yourself four hours total.
2. Build the demo, deploy it, and present it to someone for ten minutes without rehearsing.
3. Log every question they ask. Export the eval set. Write the one-page follow-up.
4. Do this three times with different domains. By the third you will have internalised the layout, the error handling and the checklist, and four hours will feel comfortable.

Checklist: before you write any code

- [ ] One user, one task, one happy path agreed and written down.
- [ ] Real (or realistically masked) sample data in hand.
- [ ] Written confirmation of whether that data may go to a hosted model API.
- [ ] Demo date, audience and the single question the demo must answer.
- [ ] Decided which parts you are deliberately stubbing, and told them.

- [ ] An end date for the prototype and what happens to it afterwards.

Checklist: code quality for a prototype that can graduate

- [ ] Business logic in `services/` with no framework imports.
- [ ] Configuration in one settings class; nothing read from `os.environ` ad hoc.
- [ ] `.env` gitignored before the first commit; `.env.example` committed.
- [ ] Dependencies pinned in a lockfile.
- [ ] Three tests: happy path, validation failure, upstream failure.
- [ ] Dockerfile built and tested on day one, not demo day.
- [ ] Structured logging of every interaction into a file you can hand over.
- [ ] A catch-all error handler; no stack trace ever reaches a user.
- [ ] Timeouts on every outbound call; a concurrency limit on the model client.
- [ ] A seed script and a visible reset button.

Checklist: demo day

- [ ] Full run-through on the presenting machine and network, two hours ahead.
- [ ] Seeded, reviewed data with nothing embarrassing in it.
- [ ] Demo cache populated; minimum replicas set to one; app warmed.
- [ ] Notifications off, sleep disabled, one browser tab, resolution tested.
- [ ] Hotspot tethered; screen recording of the happy path saved locally.
- [ ] Three improvised questions tried so you know what failure looks like.
- [ ] Stub list written down, to be stated out loud during the demo.
- [ ] A colleague briefed to carry the conversation if you need to fix something.

Checklist: after the demo

- [ ] Follow-up email within 24 hours: what was shown, what was stubbed, the questions they asked, what you recommend next.
- [ ] Eval set exported and shared.
- [ ] Debt register updated and attached, with the blockers separated from the rest.
- [ ] Teardown executed on the agreed date: app stopped, data deleted, keys rotated, confirmation sent.
- [ ] A written go/no-go recommendation, including the option of not proceeding.